

MVP2: Security Architecture, Performance Engineering & Build Pipeline

MVP2: Security Architecture, Performance Engineering & Build Pipeline

Revision: 3 **Last modified:** 2026-07-04T16:30:00Z

Revision 3 changelog: Added §8.0 "Anti-Bluff Testing Doctrine" — this document previously had zero mention of the anti-bluff evidence philosophy already in force for the Phase 0/1 control plane (docs/research/mvp/final/10-testing-acceptance-and-qa.md), a gap named in docs/research/CROSS_CUTTING_GAP_ANALYSIS.md §2.2. Added a cadence note to §8.7's CI Test Matrix clarifying "Daily"/"Weekly"/"Per release" describe local-gate cadence, not remote CI, per constitution §11.4.156 (no active .github/workflows/*.yml or .gitlab-ci.yml). **Revision 2 changelog:** Added §10 "Enterprise Hardening & Production Readiness" (staged/canary rollout across all 8 platforms, automatic rollback + rollout health monitor, privacy-first crash-reporting/telemetry pipeline, client-side update rollback for Tauri, end-to-end enterprise policy-push architecture, supply-chain release-process gates, reproducible-build release gate) — closes the cross-reference gap left by MVP2_ARCHITECTURE.md §10.1/§10.3 and MVP2_SHARED_CORE.md §5.5, both of which point here for release-process detail. Added §7.3.5 (Linux GPG package signing) to match the pipeline diagram's existing promise. Annotated the OpenVPN rows in §5.1–§5.3 as reserved/unimplemented to match helix-core's actual protocol set (OpenVPN is a reserved placeholder identifier only, never a shipped protocol). Renumbered former §10 Appendices to §11.

Document Version: 1.0.0 **Date:** 2025-07-08 **Classification:** Engineering Specification **Scope:** Comprehensive security architecture, performance optimization strategies, and build/

deployment pipeline design for the cross-platform Helix VPN client. **Sources:** Synthesized from 30+ independent web searches, academic papers, vendor documentation, and authoritative technical references. Citations use [^number^] format.

Table of Contents

- 1. [Security Architecture](#)
- 2. [Kill Switch Implementation](#)
- 3. [Leak Prevention](#)
- 4. [Obfuscation & Anti-Censorship](#)
- 5. [Performance Budgets](#)
- 6. [Performance Optimization Strategies](#)
- 7. [Build Pipeline \(CI/CD\)](#)
- 8. [Testing Strategy](#)
- 9. [Compliance & Certification](#)
- 10. [Enterprise Hardening & Production Readiness](#)
- 11. [Appendices](#)

1. Security Architecture

1.1 Threat Model

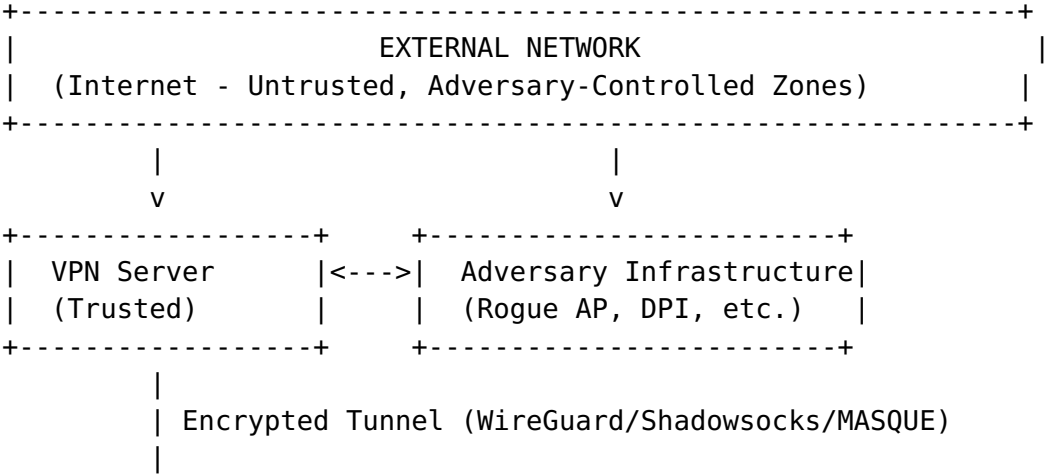
1.1.1 STRIDE Analysis for VPN Client

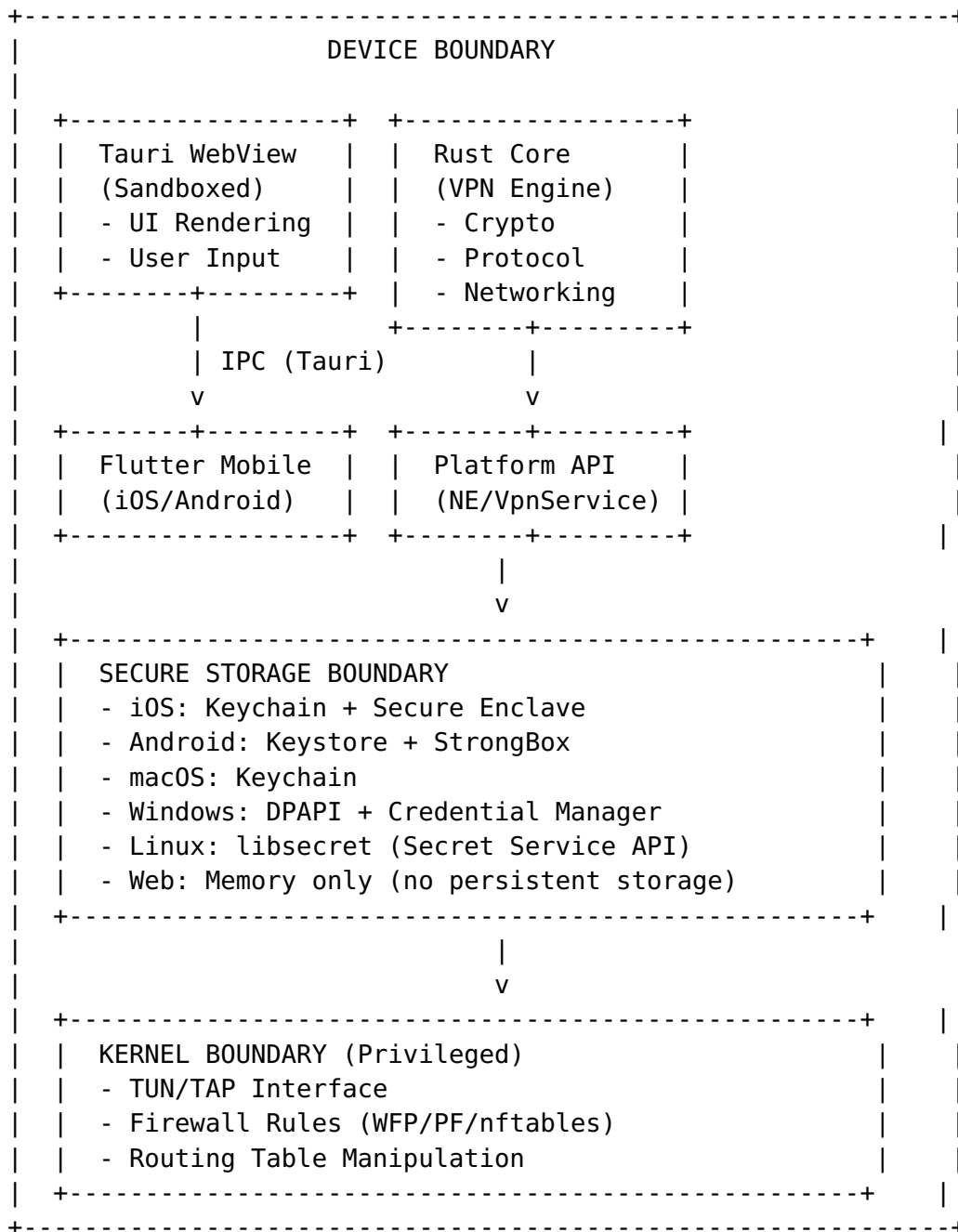
The STRIDE threat model provides a structured approach to identifying and mitigating security threats across the VPN client architecture. Each threat category is analyzed with VPN-specific examples and corresponding mitigation strategies.

Threat Category	VPN-Specific Examples	Risk Level	Mitigation Strategy
Spoofing	Fake VPN servers, rogue APs, DNS hijacking, certificate authority compromise	High	Certificate pinning (SPKI), mutual authentication, server identity verification via Ed25519 signatures
Tampering	Modified client binaries, MITM attacks, packet injection, config manipulation	High	Code signing (Ed25519), anti-tampering checks, binary integrity

Threat Category	VPN-Specific Examples	Risk Level	Mitigation Strategy
Repudiation	Denial of connection events, logging abuse, audit trail gaps	Medium	verification, W^X memory policies Client-side event logging (no PII), secure timestamping, append-only audit logs
Information Disclosure	Traffic leaks, DNS leaks, WebRTC leaks, key extraction from memory, side-channel attacks	Critical	Kill switch (fail-closed), secure key storage (Keychain/Keystore/StrongBox), memory zeroization, constant-time crypto
Denial of Service	Connection flooding, protocol-level attacks, resource exhaustion, battery drain attacks	Medium	Connection rate limiting, circuit breakers, fallback protocols, watchdog timers
Elevation of Privilege	Kernel exploit via VPN driver, privilege escalation, sandbox escape	High	Sandboxed architecture (Tauri), least-privilege design, capability-based security, seccomp-bpf on Linux

Trust Boundaries:





1.1.2 Asset Inventory

Asset Category	Specific Assets	Sensitivity	Storage Location	Lifetime
Cryptographic Keys	WireGuard private key	Critical	Secure Enclave / StrongBox / Keychain	Session + 24h rotation
Cryptographic Keys	X25519 ephemeral keys	Critical	RAM only (zeroized after use)	Per-handshake
Credentials		High	Keychain / Keystore	30 days max

Asset Category	Specific Assets	Sensitivity	Storage Location	Lifetime
Credentials	User authentication token	High	Keychain / Keystore	User-managed
	Username/ password (if used)			
Configuration	Server list + endpoints	Medium	Encrypted app storage	Persistent
Configuration	User preferences	Low	Standard app storage	Persistent
Traffic Data	Active connection metadata	High	RAM only, never logged	Ephemeral
Traffic Data	DNS queries	High	RAM only, never persisted	Ephemeral
Certificates	Server Ed25519 public keys	High	Bundled + updatable	Long-term
Certificates	CA trust anchors	Critical	System trust store	System-managed

1.1.3 Attack Surface Analysis

Attack Surface	Vector	Risk	Mitigation
Network Interface	Packet injection, spoofing	High	Authenticated encryption (ChaCha20-Poly1305), replay protection
TUN Device	Malformed packets, buffer overflow	High	Rust memory safety, input validation, bounded buffers
IPC (UI to Core)	Command injection, eavesdropping	Medium	Tauri capability-based permissions, message validation
Configuration Parser	Malformed config exploits	Medium	Strict schema validation, serde with deny_unknown_fields

Attack Surface	Vector	Risk	Mitigation
Update Mechanism	Malicious update packages	High	Ed25519 signature verification, downgrade prevention
Key Storage APIs	Side-channel extraction	Medium	Hardware-backed storage, constant-time operations
DNS Resolver	DNS hijacking, poisoning	High	DNS-over-HTTPS/TLS/QUIC, internal DNS only
WebView (Desktop)	XSS, RCE via Electron-like bugs	Low	Tauri process isolation, no Node.js runtime
Mobile Network Extension	Extension memory dump	Medium	iOS memory encryption, Android TEE isolation

1.2 Cryptographic Design

1.2.1 Key Exchange Architecture

The Helix VPN client implements a hybrid key exchange mechanism combining classical elliptic-curve cryptography with optional post-quantum protection.

Primary Key Exchange: X25519 (WireGuard Native)

"WireGuard uses Curve25519 for ECDH, Blake2s for hashing, and SipHash for hashtable keys. At ~4,000 LoC (C), it's dramatically simpler than OpenVPN (~600K LoC)."
[^59^]

Handshake Initialization:

Initiator	Responder
-- msg0: Ci, Hi, Eie (encrypted) -->	
<-- msg1: Cr, Hr, Er, Iie (enc) ---	

Where:

Ci/Cr = static Curve25519 key pairs (long-term)
Eie/Er = ephemeral Curve25519 key pairs (per-session)
Hi/Hr = protocol hashes
Iie = identity verification

Key Derivation: HKDF-SHA256(chaining_key, input_key_material)

- 3 derived keys per handshake: sending, receiving, next chaining

Optional Post-Quantum Extension: ML-KEM (Kyber) Hybrid

"In August 2024, NIST finalized three post-quantum cryptographic standards after an eight-year global evaluation process. These standards replace the mathematical foundations that quantum computers can break." [^403^]

Standard	Algorithm	Former Name	Purpose	VPN Application
FIPS 203	ML-KEM	CRYSTALS-Kyber	Key encapsulation	Hybrid with X25519
FIPS 204	ML-DSA	CRYSTALS-Dilithium	Digital signatures	Server auth (future)
FIPS 205	SLH-DSA	SPHINCS+	Backup signatures	Fallback auth

"A PQ-WireGuard handshake is less than 60% slower than a WireGuard handshake, is more than 5 times faster than an IPsec handshake using Curve25519, and more than 1000 times faster than an OpenVPN handshake." [^404^]

Hybrid Key Exchange Flow:

- When PQ mode enabled:
1. Generate X25519 ephemeral key pair (classic)
 2. Generate ML-KEM-768 key pair (post-quantum)
 3. Encapsulate shared secret using ML-KEM public key
 4. Combine X25519 shared secret + ML-KEM shared secret
 5. Derive session keys via HKDF-SHA256(combined_secret)

Performance: ~1.6x slower than classical X25519-only
Security: Protected against both classical and quantum attacks

1.2.2 Data Encryption

Cipher	Mode	Use Case	Performance	Security Level
ChaCha20-Poly1305	AEAD	Primary (WireGuard default)	Fastest on mobile/soft-CPU	256-bit
AES-256-GCM	AEAD	Fallback (hardware AES-NI)	Fastest on x86 with AES-NI	256-bit

"Use AES-256 encryption -- AES-128 is effectively weakened to 64-bit security against quantum attacks. AES-256 restores adequate security. Prefer WireGuard -- WireGuard uses ChaCha20 and Curve25519. The symmetric component (ChaCha20-Poly1305) is quantum-resistant at its key length."
[⁴⁰³]

Cipher Selection Logic:

```
fn select_cipher() -> Cipher {  
    if cpu_has_aes_ni() {  
        // AES-NI available: AES-256-GCM is fastest  
        Cipher::Aes256Gcm  
    } else {  
        // No hardware AES: ChaCha20-Poly1305 is faster  
        Cipher::ChaCha20Poly1305  
    }  
}
```

1.2.3 Authentication

Primary: Ed25519 Certificates

"x25519-dalek underwent a security audit by Quarkslab... no critical vulnerabilities found." [¹⁵⁷]

Property	Implementation
Signature scheme	Ed25519 (RFC 8032)
Key generation	Deterministic from 32-byte seed
Verification	Batch verification supported for server list
Key size	32 bytes private, 32 bytes public
Signature size	64 bytes

Optional: Username/Password Authentication

Login Flow:

1. Client generates ephemeral X25519 key pair
2. Client sends username + encrypted password (ECIES)
3. Server verifies credentials
4. Server returns short-lived JWT (24h expiry)
5. Client stores JWT in secure keychain
6. Subsequent connections use JWT + WireGuard key exchange

1.2.4 Perfect Forward Secrecy

Perfect forward secrecy is achieved through ephemeral key generation for every session:

Aspect	Implementation
Ephemeral key generation	Per-session X25519 key pair, generated client-side
Key lifetime	Ephemeral keys destroyed after handshake completion
Session key derivation	HKDF-SHA256 with ephemeral shared secret
Post-session cleanup	All session keys zeroized via zeroize crate
Compromise impact	Previous sessions remain secure even if long-term key is compromised

1.2.5 Key Rotation Schedule

Key Type	Rotation Interval	Trigger	Method
WireGuard ephemeral keys	Every session	New connection	Automatic, client-generated
WireGuard static keys	24 hours	Timer + reconnect	Silent re-key
JWT authentication tokens	24 hours	Expiry	Background refresh
Server Ed25519 certificates	On update	Server rotation	Client update mechanism
Post-quantum ML-KEM keys	Every session	New connection	Hybrid with X25519

1.3 Secure Storage

1.3.1 iOS: Keychain + Secure Enclave

"Implement certificate pinning (public key pinning or leaf certificate pinning) to mitigate MitM attacks against compromised CAs." [^428^]

// iOS Secure Storage Implementation

import Security

import LocalAuthentication

class iOSKeychainStorage {

// Store WireGuard private key in Secure Enclave

func storePrivateKey(_ key: Data, identifier: String) **throws** {
 let context = LAContext()
 }

```

context.localizedReason = "Authenticate to access VPN
keys"

let attributes: [String: Any] = [
    kSecClass as String: kSecClassGenericPassword,
    kSecAttrAccount as String: identifier,
    kSecValueData as String: key,
    kSecAttrAccessible as String:
kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly,
    kSecUseAuthenticationContext as String: context,
    kSecAccessControl as String:
SecAccessControlCreateWithFlags(
        nil,
        kSecAttrAccessibleWhenUnlockedThisDeviceOnly,
        .biometryCurrentSet,
        nil
    )!
]

SecItemAdd(attributes as CFDictionary, nil)
}

// Keychain Services for tokens and certificates
func storeToken(_ token: String, service: String) throws {
    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrService as String: service,
        kSecAttrAccount as String: "auth_token",
        kSecValueData as String: token.data(using: .utf8)!,
        kSecAttrAccessible as String:
kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly
    ]
    SecItemAdd(query as CFDictionary, nil)
}
}

```

Property	Value
Keychain access	kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly
Biometric protection	kSecAccessControlBiometryCurrentSet for VPN keys
Secure Enclave	Hardware-isolated key generation (iPhone 5s+, iPad Air+)
Data Protection	NSFileProtectionComplete for VPN configuration files
Key generation	On-device, non-exportable from Secure Enclave

1.3.2 Android: Keystore + StrongBox

"The Android Keystore system lets you store cryptographic keys in a container to make it more difficult to extract from the device. Once keys are in the Keystore, they can be used for cryptographic operations with the key material remaining non-exportable." [^433^]

```
// Android Secure Storage Implementation
import android.security.keystore.KeyGenParameterSpec
import android.security.keystore.KeyProperties
import java.security.KeyStore

class AndroidKeystoreStorage {

    private val KEYSTORE_PROVIDER = "AndroidKeyStore"

    // Generate hardware-backed key for VPN session
    fun generateVPNKeyPair(alias: String): KeyPair {
        val keyPairGenerator = KeyPairGenerator.getInstance(
            KeyProperties.KEY_ALGORITHM_EC,
            KEYSTORE_PROVIDER
        )

        val builder = KeyGenParameterSpec.Builder(
            alias,
            KeyProperties.PURPOSE_ENCRYPT or
            KeyProperties.PURPOSE_DECRYPT
        )
        .setKeySize(256)
        .setUserAuthenticationRequired(true)
        .setInvalidatedByBiometricEnrollment(true)
        .setIsStrongBoxBacked(true) // Hardware-backed when
        available
        .setRandomizedEncryptionRequired(true)

        keyPairGenerator.initialize(builder.build())
        return keyPairGenerator.generateKeyPair()
    }

    // Store authentication token securely
    fun storeAuthToken(token: String, alias: String) {
        val keyStore = KeyStore.getInstance(KEYSTORE_PROVIDER)
        keyStore.load(null)

        // Encrypt token with hardware-backed key
        val cipher = Cipher.getInstance("AES/GCM/NoPadding")
        cipher.init(Cipher.ENCRYPT_MODE, keyStore.getKey(alias,
            null))
        val encryptedToken =
            cipher.doFinal(token.toByteArray(Charsets.UTF_8))
    }
}
```

```

// Store encrypted token in EncryptedSharedPreferences
val masterKey = MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .setUserAuthenticationRequired(true)
    .build()

val sharedPreferences = EncryptedSharedPreferences.create(
    context,
    "vpn_secure_prefs",
    masterKey,

    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,

    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)
sharedPreferences.edit().putString("auth_token",
    Base64.encodeToString(encryptedToken,
    Base64.DEFAULT)).apply()
}
}

```

Property	Value
Keystore type	Android Keystore (TEE or StrongBox)
StrongBox	Dedicated secure hardware (Android 9+ devices)
Biometric binding	setUserAuthenticationRequired(true)
Enrollment invalidation	setInvalidatedByBiometricEnrollment(true)
Key purposes	PURPOSE_ENCRYPT PURPOSE_DECRYPT

1.3.3 macOS: Keychain

Storage Mechanism: macOS Keychain (SecItemAdd / SecKeychain)
 Access Level: kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly
 Modern Alternative: Secure Enclave (SecKeyGeneratePair with SE attribute)
 Migration: Automatic migration to Secure Enclave on supported hardware

1.3.4 Windows: DPAPI + Credential Manager

Primary: DPAPI (CryptProtectData / CryptUnprotectData)
 - Tied to user login credentials
 - Automatic key management by OS

Modern: Windows Hello / TPM 2.0
 - WebAuthn for hardware-backed keys
 - Platform Crypto Provider for TPM-backed storage

Credential Manager:

- CredWrite / CredRead for token storage
- CRED_TYPE_GENERIC for VPN credentials

1.3.5 Linux: libsecret (Secret Service API)

Primary: libsecret (Secret Service API / D-Bus)

- GNOME Keyring or KWallet backend
- Session-based or persistent collections

Modern: TPM 2.0 / FIDO2

- tpm2-tools for TPM-backed key operations
- libfido2 for hardware security key integration

Fallback: File-based encrypted storage

- AES-256-GCM encrypted key file
- Master password derived via Argon2id

1.3.6 Web: Memory-Only Storage

Policy: NO persistent credential storage in browser

Implementation:

- All keys stored in JavaScript memory only
- CryptoKey API with extractable: false
- sessionStorage for non-sensitive UI state only
- No localStorage for any sensitive data
- Clear all keys on page unload / visibility change
- Service Worker clears cached credentials on activate

1.3.7 Secure Storage Comparison Matrix

Platform	API	Hardware-Backed	Biometric	Non-Exportable	Encryption
iOS	Keychain + Secure Enclave	Yes (SEP)	Yes	Yes	AES-256-GCM
Android	Keystore + StrongBox	Yes (TEE/StrongBox)	Yes	Yes	AES-256-GCM
macOS	Keychain + Secure Enclave	Yes (T2/SEP)	Yes	Yes	AES-256-GCM
Windows	DPAPI + Credential Manager	Yes (TPM 2.0)	Yes (Hello)	Yes	AES-256-CBC
Linux		Optional (TPM)	No	No (software)	AES-256-GCM

Platform	API	Hardware-Backed	Biometric	Non-Exportable	Encryption
	libsecret + TPM 2.0				
Web	CryptoKey API	No	No	Partial	Memory only

2. Kill Switch Implementation

2.1 Architecture Overview

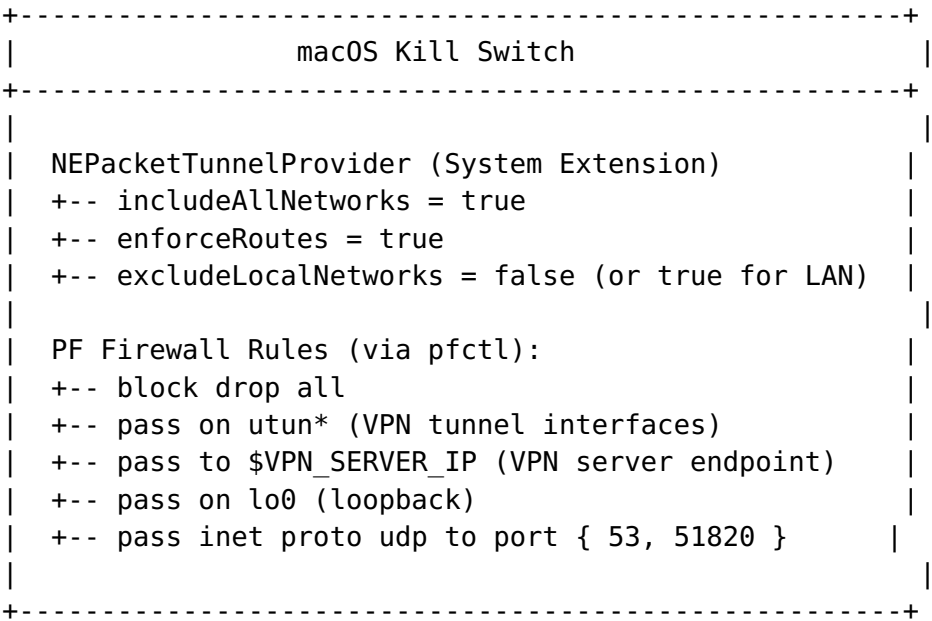
The kill switch is a critical security feature that prevents any network traffic from leaving the device when the VPN connection drops. All implementations follow the **fail-closed** design principle -- traffic is blocked by default unless explicitly allowed through the VPN tunnel.

"FreeVPN's kill switch operates at the system level, using Windows Filtering Platform on Windows, pf on macOS, and iptables/nftables on Linux." [^421^]

2.2 Platform-Specific Implementations

2.2.1 macOS: Packet Filter (PF) + NEPacketTunnelProvider

Architecture:



Implementation Details:

```
// NEPacketTunnelProvider configuration
let settings = NEPacketTunnelNetworkSettings(tunnelRemoteAddress:
    serverEndpoint)
settings.ipv4Settings = NEIPv4Settings(addresses: [tunnelIP],
    subnetMasks: ["255.255.255.0"])
settings.ipv4Settings?.includedNetworks = ["0.0.0.0/0"] // All
    traffic through tunnel
settings.ipv4Settings?.excludedRoutes = localRoutes // Exclude
    local networks if needed

// Kill switch: includeAllNetworks
settings.includeAllNetworks = true
settings.enforceRoutes = true

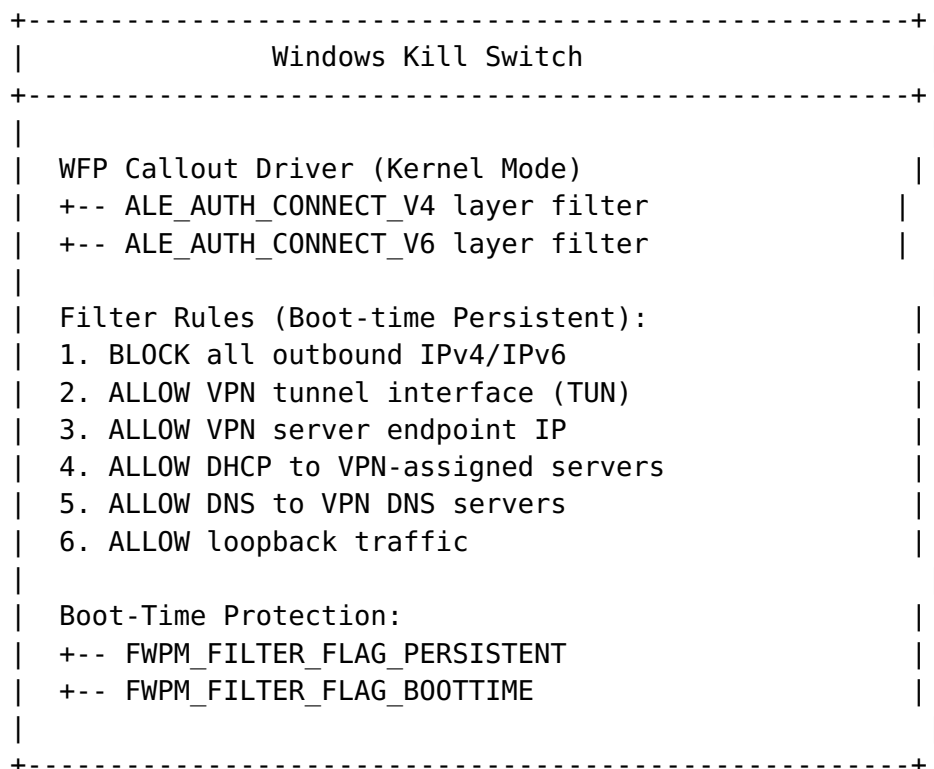
// DNS through VPN only
settings.dnsSettings = NEDNSSettings(servers: [vpnDNS])
```

Key Options:

- includeAllNetworks: Forces all traffic through tunnel [^136^]
- enforceRoutes: Ensures VPN routes take precedence over other routes
- excludeLocalNetworks: Allow AirDrop/AirPlay to bypass tunnel (optional)
- excludeCellularServices: Allow calls/messages to bypass (optional)

2.2.2 Windows: Windows Filtering Platform (WFP)

Architecture:



Critical Implementation Notes:

- WFP operates at kernel level; cannot be bypassed by user-space applications
- Filters must be registered for FWPM_LAYER_ALE_AUTH_CONNECT_V4 and FWPM_LAYER_ALE_AUTH_CONNECT_V6
- Boot-time filters persist across reboots for protection before VPN initializes
- Allow loopback traffic to prevent local service disruption
- Handle S3/S4 power state transitions by re-establishing filters on resume

```
// WFP Filter Registration (simplified)
FWPM_FILTER0 filter = {0};
filter.displayData.name = L"HelixVPN Kill Switch - Block All";
filter.layerKey = FWPM_LAYER_ALE_AUTH_CONNECT_V4;
filter.subLayerKey = HELIX_SUBLAYER_KEY;
filter.weight.type = FWP_UINT8;
filter.weight.uint8 = 0xFF; // Highest priority
filter.action.type = FWP_ACTION_BLOCK;
filter.flags = FWPM_FILTER_FLAG_PERSISTENT |
               FWPM_FILTER_FLAG_BOOTTIME;

// Allow filter for VPN tunnel
FWPM_FILTER0 allowFilter = {0};
allowFilter.displayData.name = L"HelixVPN - Allow Tunnel";
allowFilter.layerKey = FWPM_LAYER_ALE_AUTH_CONNECT_V4;
allowFilter.action.type = FWP_ACTION_PERMIT;
allowFilter.conditions[0].fieldKey =
    FWPM_CONDITION_LOCAL_INTERFACE_INDEX;
allowFilter.conditions[0].matchType = FWP_MATCH_EQUAL;
allowFilter.conditions[0].conditionValue.type = FWP_UINT32;
allowFilter.conditions[0].conditionValue.uint32 =
    tunnelInterfaceIndex;
```

2.2.3 Linux: nftables + Policy Routing

Modern nftables approach (recommended):

```
# Core kill switch rules
table inet killswitch {
    # Output chain - drop everything by default
    chain output {
        type filter hook output priority 0; policy drop;

        # Allow loopback
        oif "lo" accept

        # Allow VPN tunnel interface
        oifname "wg*" accept
        oifname "tun*" accept
        oifname "utun*" accept
    }
}
```



```

# Allow VPN server endpoint
ip daddr $VPN_SERVER_IP accept

# Allow DNS to VPN DNS only
udp dport 53 ip daddr $VPN_DNS_IP accept
tcp dport 53 ip daddr $VPN_DNS_IP accept

# Allow DHCP
udp dport { 67, 68 } accept

# Log blocked packets for debugging
log prefix "HELIX-KILLSWITCH-BLOCK: " drop
}

# Input chain
chain input {
    type filter hook input priority 0; policy drop;
    iif "lo" accept
    iifname "wg*" accept
    iifname "tun*" accept
    ct state established,related accept
}

# Forward chain
chain forward {
    type filter hook forward priority 0; policy drop;
}
}

```

Legacy iptables approach:

```

# Default drop policies
iptables -P INPUT DROP
iptables -P OUTPUT DROP
iptables -P FORWARD DROP

# Allow loopback
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

# Allow VPN tunnel interface
iptables -A OUTPUT -o tun0 -j ACCEPT
iptables -A INPUT -i tun0 -j ACCEPT

# Allow VPN establishment (DNS + WireGuard port)
iptables -A OUTPUT -p udp --dport 53 -d $VPN_DNS_IP -j ACCEPT
iptables -A OUTPUT -p udp --dport 51820 -d $VPN_SERVER_IP -j
ACCEPT

# IPv6 - drop everything unless explicitly tunneled
ip6tables -P INPUT DROP

```

```
ip6tables -P OUTPUT DROP
ip6tables -P FORWARD DROP
```

2.2.4 Android: Always-On VPN + Lockdown Mode

```
// Android Kill Switch Implementation
class HelixVpnService : VpnService() {

    override fun onStartCommand(intent: Intent?, flags: Int,
        startId: Int): Int {
        // Build VPN interface with kill switch
        val builder = Builder()
            .addAddress(tunnelIP, 24)
            .addRoute("0.0.0.0", 0) // All traffic through VPN
            .addDnsServer(vpnDNS)
            .setBlocking(true) // LOCKDOWN MODE - blocks all
            // traffic outside VPN
            .setMtu(1280)
            .allowFamily(AF_INET) // IPv4 only (block IPv6 by
            // default)
            // .allowFamily(AF_INET6) // Only enable if IPv6
            // tunneling supported

        // Establish VPN tunnel
        val interface = builder.establish()

        // Promote to foreground service
        startForeground(NOTIFICATION_ID, buildNotification())

        return START_STICKY // Auto-restart on termination
    }

    override fun onRevoke() {
        // Called when another VPN takes over
        // Kill switch remains active via setBlocking(true)
        activateEmergencyKillSwitch()
    }

    private fun activateEmergencyKillSwitch() {
        // Block all traffic until reconnection
        // Uses setBlocking(true) from VpnService.Builder
        // Prevents any traffic outside VPN tunnel
    }
}
```

Property	Value
Lockdown mode	VpnService.Builder.setBlocking(true)
Auto-restart	START_STICKY return value
Revoke handling	onRevoke() called when another VPN takes over

Property	Value
Foreground service	Required notification for persistent operation
IPv6 blocking	allowFamily(AF_INET6) omitted

2.2.5 iOS: includeAllNetworks + excludeLocalNetworks

```
// iOS Kill Switch via NEPacketTunnelProvider
class HelixPacketTunnelProvider: NEPacketTunnelProvider {

    override func startTunnel(options: [String: NSObject]?,
                               completionHandler: @escaping (Error?) -> Void) {
        let settings =
            NEPacketTunnelNetworkSettings(tunnelRemoteAddress:
            serverIP)

        // IPv4 - route all traffic through tunnel
        let ipv4Settings = NEIPv4Settings(addresses: [tunnelIP],
            subnetMasks: ["255.255.255.0"])
        ipv4Settings.includedRoutes = [NEIPv4Route.default()]
        settings.ipv4Settings = ipv4Settings

        // Kill switch: include all networks
        settings.includeAllNetworks = true
        settings.excludeLocalNetworks = false // Block local too
        // for maximum security

        // DNS through VPN only
        let dnsSettings = NEDNSSettings(servers: [vpnDNS])
        settings.dnsSettings = dnsSettings

        // Apply settings
        setTunnelNetworkSettings(settings) { error in
            completionHandler(error)
        }
    }
}
```

- includeAllNetworks = true: System automatically blocks traffic outside tunnel [^136^]
- disconnectOnSleep: Configurable (false = maintain tunnel during sleep)
- On-Demand VPN rules: Auto-reconnect on any network access attempt

2.2.6 HarmonyOS: isBlocking Flag

```
// HarmonyOS Kill Switch
// Uses VpnConfig.Builder similar to Android
VpnConfig.Builder builder = new VpnConfig.Builder();
builder.setBlocking(true); // Kill switch enabled
```

```

builder.addAddress(tunnelIP, 24);
builder.addRoute("0.0.0.0", 0);
builder.addDnsServer(vpnDNS);
builder.allowFamily(AF_INET); // IPv4 only

// HarmonyOS-specific: isBlocking flag
// Requires com.huawei.permission.SECUREVpnConfig permission
VpnConfig config = builder.build();

```

2.3 Activation Triggers

Trigger	Detection Method	Response Time	Action
VPN connection drops	Tunnel interface down / keepalive timeout	< 50ms	Activate kill switch
Network change	System network change notification	< 100ms	Kill switch ON during reconnect
Sleep/wake	Power state change notification	< 100ms	Re-establish tunnel or kill switch
Airplane mode	Radio state change	< 100ms	Kill switch ON until VPN reconnects
WiFi <-> Cellular handoff	Network path change	< 100ms	Brief kill switch during handoff
Another VPN takes over	onRevoke() (Android) / NE callback	< 50ms	Kill switch ON immediately
Process crash	Watchdog / heartbeat timeout	< 200ms	Kill switch ON (OS-level persistence)

2.4 Response Time Requirements

Provider	Kill Switch Response Time	Measurement Method
Proton VPN, Mullvad, IVPN	87-112 ms	tcpdump + kernel timestamping [^398^]
Helix VPN Target	< 100 ms	Kernel-level filtering
Industry average	100-500 ms	Varies by implementation
	1-2+ seconds	

Provider	Kill Switch Response Time	Measurement Method
Poor implementations		User-space packet filtering

Response Time Budget Breakdown:

Total Target: < 100ms

- VPN disconnect detection: < 20ms (keepalive timeout / interface monitoring)
- Kill switch activation: < 30ms (WFP filter commit / nftables reload / system API)
- Rule propagation: < 40ms (kernel rule installation, network stack update)
- Buffer clearance: < 10ms (drop any in-flight packets)

2.5 Recovery After VPN Reconnect

Recovery Flow:

1. VPN disconnect detected -> Kill switch ACTIVATED (< 100ms)
2. Reconnection process begins
 - a. Resolve VPN server DNS
 - b. Perform WireGuard handshake
 - c. Configure tunnel interface
 - d. Verify tunnel connectivity (ping through tunnel)
3. Tunnel verified -> Kill switch DEACTIVATED
4. Normal traffic flow resumes

Edge Cases:

- Reconnection fails after N retries -> Kill switch remains ON
- User manually disconnects -> Kill switch OFF (user choice)
- Multiple network changes -> Kill switch ON throughout
- Server unreachable -> Kill switch ON, try fallback servers

3. Leak Prevention

3.1 DNS Leak Prevention

"Launch Surfshark on a phone, laptop, or smart TV and it silently swaps your OS to Surfshark-owned DNS, blocks IPv6, and flips on a kill switch; no setup screens required. We stress-tested ten servers across North America, Europe, and Asia. Every packet capture showed zero DNS or IPv6 leaks." [^406^]

Multi-Layer DNS Leak Prevention:

Layer	Technique	Implementation	Effectiveness
1. Custom DNS Assignment	Push VPN DNS via DHCP/tunnel options	dnsSettings (iOS), addDnsServer (Android), DHCP option 6	Base layer
2. Firewall-based Interception	Block all UDP/53 except to VPN DNS	WFP filter (Win), nftables rule (Linux), PF rule (macOS)	Critical
3. DNS-over-HTTPS (DoH)	Encrypted DNS via HTTPS	Cloudflare DoH, custom resolver	High
4. DNS-over-TLS (DoT)	Encrypted DNS channel	Port 853 TLS	High
5. DNS-over-QUIC (DoQ)	QUIC-encapsulated DNS	Fastest encrypted option	High
6. IPv6 DNS Blocking	Block AAAA queries or IPv6 entirely	Drop IPv6 DNS at firewall	Critical

Platform-Specific DNS Enforcement:

iOS: NEDNSSettings - exclusive DNS through tunnel
Android: VpnService.Builder.addDnsServer() + firewall rules
macOS: scutil --dns override + PF rules
Windows: SetInterfaceDnsSettings + WFP DNS interception
Linux: resolvconf update + nftables DNS redirection

3.2 WebRTC Leak Prevention

"WebRTC is a technology that allows browsers to communicate directly with each other, but it can also expose your real IP address even when you're using a VPN."
[^402^]

Approach	Implementation	Platform
Disable WebRTC entirely	media.peerconnection.enabled = false (Firefox)	Browser extension
WebRTC Network Limiter	Force TURN relay through VPN	Chrome extension

Approach	Implementation	Platform
System-level block	Route all traffic through tunnel (includeAllNetworks)	All platforms
Tauri WebView	Disable WebRTC via WebView preferences if not needed	Desktop app
In-app browser	Block WebRTC at application level	Mobile apps

WebRTC Leak Test Verification:

- Connect to VPN
- Visit browserleaks.com/webrtc
- Verify only VPN-assigned IP is visible
- Ensure no local/internal IPs are exposed

3.3 IPv6 Leak Handling

"If your VPN provider doesn't explicitly support IPv6, look for options to block IPv6 traffic in your VPN's settings. Most modern VPN applications now include this feature to prevent IPv6 leaks." [^402^]

Approach	Method	Security Level	Recommendation
IPv6 blocking	Disable IPv6 stack entirely	Highest	Recommended default
IPv6 through tunnel	Route all IPv6 through VPN tunnel	High	If VPN supports IPv6
IPv6 local only	Block external IPv6, allow link-local	Medium	Compromise option
No handling	IPv6 traffic bypasses VPN	Vulnerable	Never use

Platform-Specific IPv6 Disabling:

Windows

```
Get-NetAdapterBinding | Disable-NetAdapterBinding -ComponentID ms_tcpip6
```

macOS

```
networksetup -setv6off Wi-Fi
networksetup -setv6off Ethernet
```

Linux

```
sysctl -w net.ipv6.conf.all.disable_ipv6=1
```

```
sysctl -w net.ipv6.conf.default.disable_ipv6=1
```

```
# iOS: Blocked automatically by includeAllNetworks
# Android: Block via VpnService.Builder.allowFamily(AF_INET6) =
        false
```

3.4 Time Zone Leak Prevention

Vector	Risk	Mitigation
System time zone	Leaks geographic location	Option to spoof time zone to match VPN exit
JavaScript Intl.DateTimeFormat	Web page can detect time zone	Browser extension to override
HTTP Date header	Server logs show time zone	Not directly leakable via VPN
NTP queries	Unencrypted time sync leaks location	Route NTP through VPN or disable

Implementation:

- Detect VPN exit server time zone
- Offer option to sync system time zone to match
- Block NTP (UDP 123) outside VPN tunnel
- Route NTP through VPN tunnel if time sync required

3.5 Geolocation API Spoofing

API	Leak Vector	Mitigation
navigator.geolocation	GPS/WiFi-based precise location	Browser extension: deny or spoof to VPN exit
WiFi network name (SSID)	Unique identifier	Never expose outside VPN tunnel
Bluetooth beacons	Location tracking	Block Bluetooth during VPN use (optional)
Cell tower info	Precise geographic location	Not controllable by VPN app (OS level)

4. Obfuscation & Anti-Censorship

4.1 Protocol Selection Guide

"WireGuard is famous for its minimal codebase, exceptional performance, and modern cryptography. However, the WireGuard protocol is transparently designed with a fixed packet format and lacks built-in traffic obfuscation. Its unique handshake pattern and consistent packet structure make it relatively easy for DPI to identify and block via pattern matching." [^397^]

Censorship Level	Recommended Protocol	Configuration
None / Light	WireGuard (direct)	Default ports, dynamic port hopping
Moderate	Shadowsocks + simple-obfs	AEAD-2022 ciphers, WebSocket transport
Heavy	V2Ray/Xray (VLESS+Vision+REALITY)	TLS fingerprint cloning, XTLS
Extreme	MASQUE (HTTP/3 over QUIC)	Blends with normal HTTPS traffic

"For Light Censorship Environments: For ultimate speed and low latency, choose WireGuard (coupled with dynamic ports). For Moderate Censorship Environments: For a balance of speed and stealth, Shadowsocks with simple-obfs or V2Ray (WebSocket+TLS) are reliable choices. For Heavy Censorship Environments (e.g., China, Iran): Highly camouflaged protocols are essential. V2Ray/Xray (VLESS+Vision+Reality) or Trojan-Go are currently among the most effective solutions." [^397^]

4.2 Shadowsocks SIP022 AEAD-2022

"Shadowsocks 2022 is a secure proxy protocol for TCP and UDP traffic. The protocol uses AEAD with a pre-shared symmetric key to protect payload integrity and confidentiality." [^156^]

Key Features of SIP022 AEAD-2022:

- Full replay protection (mandatory, not optional)
- Session-based UDP proxying
- Session subkey derivation with BLAKE3

- TCP: length-chunk-payload-chunk model with headers
- UDP: separate header encryption + AEAD body
- Supported ciphers: 2022-blake3-aes-128-gcm, 2022-blake3-aes-256-gcm, 2022-blake3-chacha20-poly1305

```
// Shadowsocks AEAD-2022 cipher selection
fn select_aead_2022_cipher(method: &str, psk: &[u8]) -> Box<dyn
    Cipher> {
    match method {
        "2022-blake3-aes-256-gcm" => {
            Box::new(AEAD2022Cipher::new(CipherMethod::Aes256Gcm,
                psk))
        }
        "2022-blake3-chacha20-poly1305" => {
            Box::new(AEAD2022Cipher::new(CipherMethod::ChaCha20Poly1305,
                psk))
        }
        _ => panic!("Unsupported AEAD-2022 cipher: {}", method),
    }
}
```

4.3 V2Ray VMess + XTLS Vision

"V2Ray and its fork Xray represent a class of modular, highly configurable proxy platforms. Their core transport protocols, VMess and VLESS, feature built-in dynamic port allocation, metadata obfuscation, and optional full TLS encapsulation."
[³⁹⁷]

XTLS Vision Flow Control:

- Identifies and directly transmits application data after TLS handshake
- Avoids double-encryption overhead (TLS + proxy encryption)
- Reduces latency by skipping unnecessary encryption layers
- Only encrypts proxy metadata, not the already-encrypted TLS payload

XTLS Vision Mode:

```
Client -> Server: TLS ClientHello (direct, not re-encrypted)
Server -> Client: TLS ServerHello (direct)
[TLS handshake completes directly]
Application data flows through without double encryption
Only proxy framing/metadata is encrypted by VLESS
```

4.4 REALITY Protocol

REALITY is a TLS fingerprint cloning technique that makes VPN traffic indistinguishable from normal HTTPS connections to popular websites.

"REALITY protocol: 'borrows' TLS certificate fingerprints from popular websites." [^397^]

How REALITY Works:

- 1. Client connects to VPN server
- 2. Server presents TLS certificate that mimics a popular website (e.g., www.microsoft.com)
- 3. TLS fingerprint matches the popular site exactly
- 4. DPI sees normal HTTPS traffic to a legitimate domain
- 5. Only the client and server know the actual destination

REALITY Configuration:

```
dest: www.microsoft.com:443 # Target to mimic
xver: 1                      # Proxy protocol version
serverNames:                 # Allowed SNI values
  - www.microsoft.com
  - outlook.com
privateKey: <Ed25519 private key>
shortIds: ["", "0123", "4567"] # Short ID variants
```

4.5 DAITA (Defense Against AI-based Traffic Analysis)

"GotaTun... DAITA (Defense Against AI-guided Traffic Analysis) integration." [^158^]

DAITA Implementation:

- Pads packets to uniform sizes to prevent traffic pattern analysis
- Injects dummy traffic to mask real communication patterns
- Randomizes packet timing to defeat timing analysis
- Adapts padding strategy based on detected analysis sophistication

DAITA Protection Layers:

- 1. Packet Size Padding: All packets padded to MTU or fixed sizes
- 2. Dummy Traffic Injection: Background noise packets at random intervals
- 3. Timing Obfuscation: Randomized inter-packet delays
- 4. Flow Shape Masking: Multiple flows blended together

4.6 Domain Fronting

Aspect	Status
Mechanism	Route traffic through CDN edge nodes with different SNI and Host headers
Effectiveness	Largely blocked by major CDNs since 2018-2020

Aspect	Status
CloudFront	Blocked - requires matching SNI and Host
CloudFlare	Blocked - enforces SNI consistency
Fastly	Blocked - requires matching headers
Recommendation	Not recommended for modern VPN implementations

4.7 Multi-Hop / Cascading Connections

Multi-hop Architecture:

Standard Single-Hop:

[Client] =====> [VPN Server] =====> [Internet]
(Your IP visible to server only)

Multi-Hop (Entry-Exit):

[Client] =====> [Entry Server] =====> [Exit Server] =====> [Internet]
(Entry sees your IP, not destination)
(Exit sees destination, not your IP)
(No single server has complete picture)

Implementation Approaches:

Approach	Description	Latency Impact	Security Benefit
WireGuard routing	Route-based chaining using AllowedIPs	Medium (+1 RTT)	Entry-exit separation
MASQUE dual-hop	Apple iCloud Private Relay style	Medium	Standardized, proven
Trojan relay chain	Entry + relay nodes	High	Built-in multi-hop
Jurisdictional routing	Entry in privacy country, exit in target country	Medium	Legal separation

5. Performance Budgets

Protocol implementation status note (added Revision 2): helix-core natively implements WireGuard, Shadowsocks (SIP022 AEAD-2022), V2Ray/VMess+XTLS+REALITY, and MASQUE/QUIC. **OpenVPN is a reserved, unimplemented placeholder protocol identifier only** in helix-core's

protocol-negotiation enum — no OpenVPN client or server implementation ships in MVP2. Every OpenVPN row below is retained either as (a) an explicit industry-reference comparison point (§5.4, §5.5 — clearly labeled "Reference Data") or (b) marked **"reserved, not implemented"** where it previously appeared alongside genuinely-shipped protocol targets (§5.1–§5.3), so it is never mistaken for a Helix VPN performance commitment.

5.1 Cross-Platform Performance Targets

Metric	Desktop Target	Mobile Target	Web Target
Cold startup	< 2s	< 3s	< 1s
Connection time	< 500ms	< 1s	< 2s
Memory footprint	< 100MB	< 80MB	< 50MB
Bundle size	< 15MB	< 25MB	< 5MB
Throughput (WireGuard)	> 500 Mbps	> 100 Mbps	> 10 Mbps
Throughput (OpenVPN — reserved, not implemented)	> 200 Mbps	> 50 Mbps	> 5 Mbps
CPU usage (idle)	< 1%	< 1%	< 1%
CPU usage (500 Mbps)	< 25%	< 40%	N/A
Battery impact (hourly)	N/A	< 3%/hour	N/A
Kill switch response	< 100ms	< 200ms	N/A
Reconnection time	< 2s	< 3s	< 2s
Latency overhead	< 1ms	< 2ms	< 5ms

5.2 Protocol-Specific Performance Targets

Protocol	Desktop Throughput	Mobile Throughput	Latency Overhead	Connection Time
WireGuard	> 500 Mbps	> 100 Mbps	< 1ms	< 100ms
Shadowsocks	> 400 Mbps	> 80 Mbps	< 2ms	< 200ms
OpenVPN (UDP) — reserved, not implemented	> 200 Mbps	> 50 Mbps	< 5ms	< 2s
IKEv2/IPsec	> 300 Mbps	> 80 Mbps	< 3ms	< 1s
MASQUE/QUIC	> 400 Mbps	> 100 Mbps	< 2ms	< 200ms

5.3 Detailed Platform Performance Budgets

Metric	Windows	macOS	Linux	iOS	Android	Web
Throughput (WG)	500+ Mbps	500+ Mbps	500+ Mbps	100+ Mbps	100+ Mbps	10+ Mbps
Throughput (OVPN — reserved, not implemented)	200+ Mbps	200+ Mbps	200+ Mbps	50+ Mbps	50+ Mbps	5+ Mbps
Connection (WG)	< 100ms	< 100ms	< 100ms	< 200ms	< 200ms	< 2s
Connection (OVPN — reserved, not implemented)	< 2s	< 2s	< 2s	< 3s	< 3s	< 5s
CPU idle	< 1%	< 1%	< 1%	< 2%	< 2%	< 1%
CPU (500 Mbps)	< 25%	< 25%	< 20%	< 40%	< 40%	N/A
Memory	< 100MB	< 100MB	< 80MB	< 50MB	< 50MB	< 50MB
Battery/hr	N/A	N/A	N/A	< 5%	< 5%	N/A
Kill switch	< 100ms	< 100ms	< 100ms	< 200ms	< 200ms	N/A
App launch	< 2s	< 2s	< 2s	< 2s	< 2s	< 1s
Reconnect	< 2s	< 2s	< 2s	< 3s	< 3s	< 2s
Binary size	< 15MB	< 15MB	< 15MB	< 25MB	< 25MB	< 5MB

5.4 Protocol Benchmarks (Reference Data)

Protocol	Single-Thread TCP	Multi-Thread TCP	Latency Overhead	Initiation Time
WireGuard	892 Mbps	~line rate	+0.8 ms	6.9 ms
IPsec (IKEv2)	655 Mbps	~900 Mbps	+2.1 ms	32 ms
OpenVPN (UDP)	412 Mbps	~600 Mbps	+5.5 ms	~1000 ms

Source: University of Amsterdam benchmarks [^423^],
Cloud environment tests [^420^]

5.5 Battery Impact Budgets (Mobile)

"WireGuard's codebase is roughly 15-25 times smaller than OpenVPN's, its cryptographic operations are faster, and its connection design allows the tunnel to sleep cleanly between packets. Real-world tests consistently show 20-30% lower battery consumption with WireGuard compared to OpenVPN under equivalent conditions." [^419^]

Scenario	WireGuard	OpenVPN	Target
Active streaming (1 hour)	~8-12%	~12-18%	< 10%
Idle connected (1 hour)	~2-4%	~4-8%	< 3%
Background sync (1 hour)	~3-5%	~5-10%	< 4%
Reconnection cycle	~0.5-1%	~1-2%	< 1%

6. Performance Optimization Strategies

6.1 Rust Zero-Copy Packet Handling

"Zero-copy reduces memory allocations, CPU cycles, and improves CPU cache utilization, leading to better performance, especially with large data sets." [^445^]

```
// Zero-Copy Packet Processing Pipeline
use bytes::{Bytes, BytesMut};
use zerocopy::{AsBytes, FromBytes, FromZeroes};

// 1. Reference-counted buffers via `bytes` crate
// Multiple packet handlers share the same buffer without copying
fn process_packet_zero_copy(buffer: Bytes) -> Result<()> {
    // Parse packet header without copying
    let header = parse_header(&buffer[..HEADER_SIZE]);

    // Encrypt payload in place (no allocation)
    let (header, payload) = buffer.split_at(HEADER_SIZE);

    // Route to multiple handlers with shared references
    tunnel.write(&buffer)?; // Send through tunnel (ref count +1)
    stats.record(&buffer); // Stats collection (ref count +1)
    // Buffer freed when last reference dropped
    Ok(())
}

// 2. Structured network packets with zerocopy crate
#[derive(AsBytes, FromBytes, FromZeroes)]
```

```
#[repr(C)]
struct WireGuardPacket {
    packet_type: u8,
    reserved: u8,
    receiver_index: u32,
    sender_index: u32,
    ephemeral_key: [u8; 32],
    encrypted_static: [u8; 32],
    encrypted_timestamp: [u8; 12],
    mac1: [u8; 16],
    mac2: [u8; 16],
}

// 3. Object pooling for packet buffers
// Pre-allocate buffer pool to avoid runtime allocation
lazy_static! {
    static ref BUFFER_POOL: Arc<Mutex<Vec<BytesMut>>> =
        Arc::new(Mutex::new(Vec::with_capacity(1024)));
}

fn acquire_buffer() -> BytesMut {
    BUFFER_POOL.lock().unwrap().pop()
        .unwrap_or_else(|| BytesMut::with_capacity(1500))
}

fn release_buffer(buf: BytesMut) {
    BUFFER_POOL.lock().unwrap().push(buf);
}
```

"The bytes crate facilitates zero-copy network programming by allowing multiple Bytes objects to point to the same underlying memory. This is managed by using a reference count to track when the memory is no longer needed and can be freed." [^443^]

6.2 Lock-Free Data Structures

```
// Lock-free channels for packet routing
use crossbeam::channel::{bounded, unbounded};
use crossbeam_queue::ArrayQueue;

// Packet queue between TUN reader and encryptor
static PACKET_QUEUE: ArrayQueue<Bytes> = ArrayQueue::new(1024);

// Lock-free statistics counters
use std::sync::atomic::{AtomicU64, Ordering};

static PACKETS_SENT: AtomicU64 = AtomicU64::new(0);
static BYTES_SENT: AtomicU64 = AtomicU64::new(0);

fn record_send(bytes: usize) {
```



```
PACKETS_SENT.fetch_add(1, Ordering::Relaxed);
BYTES_SENT.fetch_add(bytes as u64, Ordering::Relaxed);
}
```

6.3 Tokio Async Runtime Tuning

"Futures are zero-cost, meaning they do not create any overhead. It costs nothing for a program to use Futures, compared to OS threads, which are expensive. Futures are stackless." [^414^]

```
// Tokio runtime optimized for VPN workload
fn create_vpn_runtime() -> Runtime {
    tokio::runtime::Builder::new_multi_thread()
        .worker_threads(num_cpus::get()) // Use all cores for
        .max_blocking_threads(512)       // For blocking I/O
        .thread_stack_size(2 * 1024 * 1024) // 2MB stack per
        .enable_all()                    // Enable I/O, timer,
        .event_interval(61)              // Batch events for
        .global_queue_interval(61)       // Work-stealing
        .max_io_events_per_tick(1024)    // Batch process I/O
        .build()
        .unwrap()
}

// Tokio features for VPN:
// - epoll/kqueue/IOCP-based async I/O (automatic platform
//   selection)
// - Multi-threaded work-stealing scheduler
// - Timer support for keepalive intervals
// - UDP/TCP socket support for WireGuard and obfuscation
tokio::net::UdpSocket; // WireGuard transport
tokio::time::interval; // Keepalive timer
tokio::spawn;          // Per-connection tasks
tokio::select!;        // Cancellation and timeout handling
```

Tokio Tuning Parameters:

Parameter	Default	VPN-Optimized	Rationale
worker_threads	4	num_cpus::get()	Maximize parallelism
event_interval	61	61	Good throughput default

Parameter	Default	VPN-Optimized	Rationale
max_io_events_per_tick	1024	1024	Batch I/O processing
thread_stack_size	2MB	2MB	Sufficient for crypto operations
max_blocking_threads	512	512	Allow many blocking operations

6.4 Connection Pooling and Keepalive Optimization

```
// Adaptive keepalive based on network conditions
struct KeepaliveConfig {
    base_interval: Duration,
    network_type: NetworkType,
    current_rtt: Duration,
}

enum NetworkType {
    WiFi,
    Cellular4G,
    Cellular5G,
    Metered,
}

impl KeepaliveConfig {
    fn interval(&self) -> Duration {
        match self.network_type {
            NetworkType::WiFi => Duration::from_secs(25),      //
            // NAT timeout ~30s
            NetworkType::Cellular4G =>
            Duration::from_secs(15), // Shorter for cellular
            NetworkType::Cellular5G => Duration::from_secs(20),
            NetworkType::Metered =>
            Duration::from_secs(60),    // Conservative on metered
        }
    }

    fn adaptive_interval(&self) -> Duration {
        // Adjust based on RTT: longer keepalive for stable
        // connections
        let base = self.interval();
        if self.current_rtt < Duration::from_millis(50) {
            base + Duration::from_secs(5) // Stable: can wait
            longer
        } else {
            base // Unstable: use default
        }
    }
}
```

```

    }
}

```

6.5 Adaptive MTU Discovery

```

// Path MTU discovery for VPN tunnel
struct MtuDiscovery {
    current_mtu: u16,
    min_mtu: u16,      // 1280 (IPv6 minimum)
    max_mtu: u16,      // 1500 (Ethernet) or 9000 (Jumbo)
    probe_state: ProbeState,
}

impl MtuDiscovery {
    fn discover(&mut self) -> u16 {
        // Binary search for optimal MTU
        // Account for WireGuard overhead (32 bytes header + 16
        // bytes auth tag)
        // Account for UDP/IP headers (28 bytes IPv4, 48 bytes
        // IPv6)
        let wg_overhead = 32 + 16 + 28; // = 76 bytes for IPv4
        let effective_mtu = self.current_mtu - wg_overhead;

        // Send probe packets of decreasing size
        // Detect ICMP "Fragmentation Needed" or timeout
        // Adjust tunnel MTU accordingly

        self.current_mtu
    }
}

```

6.6 Batch Packet Processing

```

// Batch process multiple packets in single system call
fn batch_process_packets(tun: &TunDevice, socket: &UdpSocket) ->
    io::Result<()> {
    let mut batch = [MaybeUninit::<Bytes>::uninit(); 32];

    // Read multiple packets from TUN in one go
    let count = tun.readv(&mut batch)?;

    // Process all packets
    for i in 0..count {
        let packet = unsafe { batch[i].assume_init_ref() };

        // Encrypt and send
        let encrypted = encrypt_packet(packet)?;
        socket.send(&encrypted)?;
    }
}

```

```
    Ok(())
}
```

6.7 Memory-Mapped TUN I/O

```
// Memory-mapped TUN for zero-copy I/O (Linux)
#[cfg(target_os = "linux")]
fn setup_mmap_tun(tun_fd: RawFd) -> io::Result<MmapTun> {
    use memmap2::{MmapMut, MmapOptions};

    // Map TUN ring buffer into process memory
    let mmap = unsafe {
        MmapOptions::new()
            .len(16 * 1024 * 1024) // 16MB ring buffer
            .map_mut(&tun_fd)?
    };

    Ok(MmapTun { mmap })
}

// Platform-specific optimizations:
// Linux: io_uring for async TUN I/O (Linux 5.1+)
// macOS: NEPacketTunnelProvider with batched reads
// Windows: Registered I/O (RIO) for WinTun
// iOS: NEPacketTunnelProvider batching
// Android: VpnService with buffered FileChannel
```

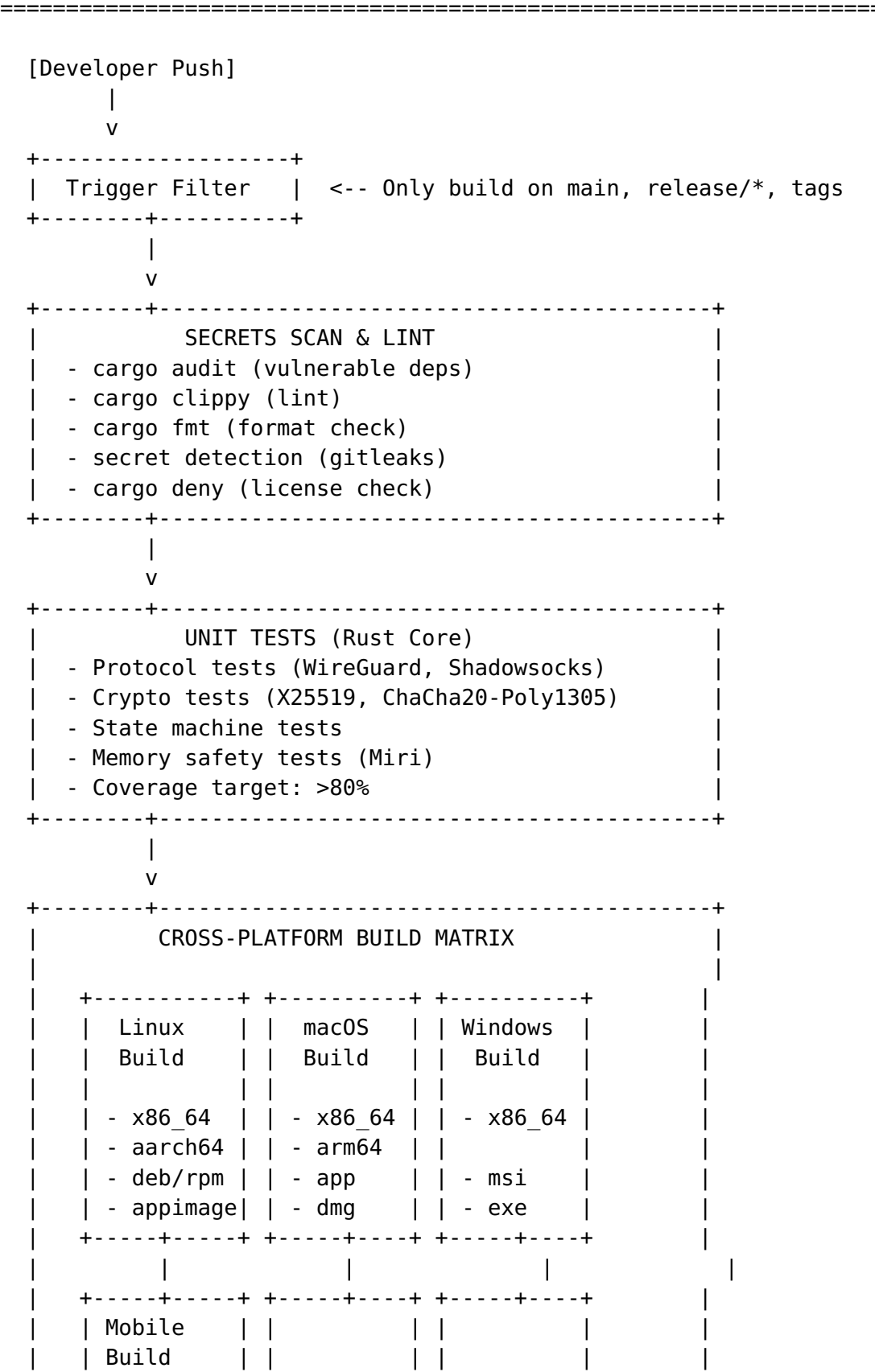
6.8 Performance Summary: Key Optimizations

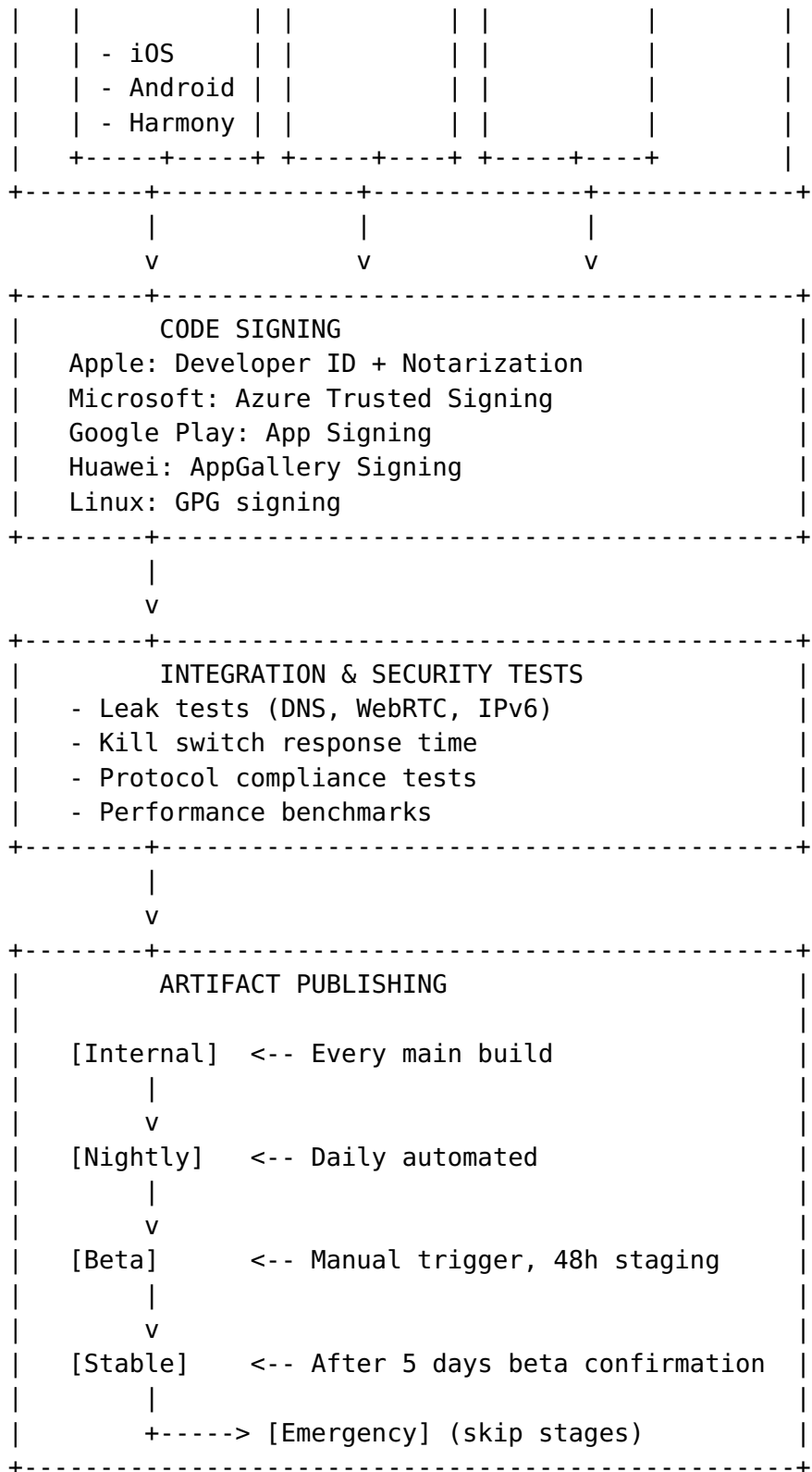
Optimization	Expected Improvement	Implementation Complexity
Zero-copy packet handling	15-30% throughput	Medium
Lock-free data structures	10-20% latency reduction	Medium
Tokio runtime tuning	5-15% CPU reduction	Low
Adaptive MTU	5-10% throughput	Low
Batch packet processing	20-40% syscall reduction	Medium
Memory-mapped TUN I/O	10-25% throughput (Linux)	High
Connection pooling	5-10% connection time	Medium
Battery-optimized keepalive	20-30% battery savings	Medium

7. Build Pipeline (CI/CD)

7.1 GitHub Actions Workflow Architecture

CI/CD PIPELINE OVERVIEW





7.2 Multi-Platform Build Matrix

```
# .github/workflows/build.yml
```

```
name: Cross-Platform Build
```

```
on:
```

```
  push:
```

```
branches: [main, release/*]
tags: ['v*']
pull_request:
  branches: [main]
```

jobs:

Phase 1: Lint and Security Scan

lint-and-scan:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: dtolnay/rust@stable
- run: cargo fmt -- --check
- run: cargo clippy --all-targets --all-features -- -D warnings
- run: cargo audit
- run: cargo deny check
- uses: trufflesecurity/trufflehog@main

Phase 2: Unit Tests

unit-tests:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: dtolnay/rust@stable
- run: cargo test --all-features --workspace
- run: cargo test --all-features --workspace -- --ignored
- run: | *# Miri for memory safety*
rustup component add miri
cargo miri test

Phase 3: Cross-Platform Builds

build-desktop:

needs: [lint-and-scan, unit-tests]

strategy:

fail-fast: false

matrix:

include:

macOS Universal Binary (x86_64 + arm64)

- os: macos-latest
target: universal-apple-darwin
arch: x86_64,arm64
artifact: HelixVPN.dmg

Windows x86_64

- os: windows-latest
target: x86_64-pc-windows-msvc
arch: x86_64
artifact: HelixVPN.msi

Linux x86_64

- os: ubuntu-latest

```

    target: x86_64-unknown-linux-gnu
    arch: x86_64
    artifact: helix-vpn.AppImage

# Linux aarch64 (cross-compile)
- os: ubuntu-latest
  target: aarch64-unknown-linux-gnu
  arch: aarch64
  cross: true
  artifact: helix-vpn-aarch64.AppImage

runs-on: ${{ matrix.os }}
steps:
  - uses: actions/checkout@v4
  - uses: dtolnay/rust@stable

  - name: Install cross-compilation tools
    if: matrix.cross
    run: cargo install cross

  - name: Build (native)
    if: '!matrix.cross'
    run: cargo build --release --target ${{ matrix.target }}

  - name: Build (cross)
    if: matrix.cross
    run: cross build --release --target ${{ matrix.target }}

build-mobile:
  needs: [lint-and-scan, unit-tests]
  strategy:
    fail-fast: false
  matrix:
    include:
      # iOS (arm64 + x86_64 simulator)
      - platform: ios
        runs-on: macos-latest
        targets: aarch64-apple-ios,x86_64-apple-ios,aarch64-apple-ios-sim
        artifact: HelixVPN.ipa

      # Android (arm64-v8a, armeabi-v7a, x86_64)
      - platform: android
        runs-on: ubuntu-latest
        targets: aarch64-linux-android,armv7-linux-androideabi,x86_64-linux-android
        artifact: app-release.aab

      # HarmonyOS (arm64)
      - platform: harmonyos
        runs-on: ubuntu-latest
        targets: aarch64-unknown-linux-gnu

```



```

    artifact: helix-vpn.hap

runs-on: ${ matrix.runs-on }
steps:
  - uses: actions/checkout@v4

  - name: Setup Rust targets
    run: rustup target add ${ matrix.targets }

  - name: Build iOS Framework
    if: matrix.platform == 'ios'
    run: |
      cargo build --release --target aarch64-apple-ios
      cargo build --release --target x86_64-apple-ios
      cargo build --release --target aarch64-apple-ios-sim
      # Create XCFramework
      xcodebuild -create-xcframework \
        -library target/aarch64-apple-ios/release/libhelix.a \
        -library target/aarch64-apple-ios-sim/release/
        libhelix.a \
        -output HelixVPNCore.xcframework

  - name: Build Android Libraries
    if: matrix.platform == 'android'
    run: |
      cargo ndk -t armeabi-v7a -t arm64-v8a -t x86_64 build --
      release

  - name: Build HarmonyOS
    if: matrix.platform == 'harmonyos'
    run: |
      # Using HarmonyOS SDK cross-compilation
      cargo build --release --target aarch64-unknown-linux-gnu

```

7.3 Code Signing

7.3.1 Apple: Developer ID + Notarization

"Pass CSC_LINK and CSC_KEY_PASSWORD directly -- electron-builder creates and manages a temporary keychain automatically." [^410^]

```

# macOS Code Signing & Notarization
sign-macos:
  runs-on: macos-latest
  needs: build-desktop
  steps:
    - name: Download macOS artifact
      uses: actions/download-artifact@v4

    - name: Import signing certificate

```

```

env:
  MACOS_CERTIFICATE: ${ secrets.MACOS_CERTIFICATE_P12 }}
  MACOS_CERTIFICATE_PWD: $
  {{ secrets.MACOS_CERTIFICATE_PASSWORD }}
  KEYCHAIN_PASSWORD: ${ secrets.KEYCHAIN_PASSWORD }}
run: |
  echo "$MACOS_CERTIFICATE" | base64 --decode >
  certificate.p12
  security create-keychain -p "$KEYCHAIN_PASSWORD"
  build.keychain
  security default-keychain -s build.keychain
  security unlock-keychain -p "$KEYCHAIN_PASSWORD"
  build.keychain
  security import certificate.p12 -k build.keychain -P
  "$MACOS_CERTIFICATE_PWD" -T /usr/bin/codesign
  security set-key-partition-list -S apple-
  tool:,apple:,codesign: -s -k "$KEYCHAIN_PASSWORD"
  build.keychain

- name: Sign application
  run: |
    codesign --force --options runtime --sign "Developer ID
    Application: Helix VPN Inc" \
    --entitlements entitlements.plist HelixVPN.app

- name: Notarize
  env:
    APPLE_ID: ${ secrets.APPLE_ID }}
    APPLE_APP_SPECIFIC_PASSWORD: $
    {{ secrets.APPLE_APP_SPECIFIC_PASSWORD }}
    APPLE_TEAM_ID: ${ secrets.APPLE_TEAM_ID }}
  run: |
    ditto -c -k --keepParent HelixVPN.app HelixVPN.zip
    xcrun notarytool submit HelixVPN.zip \
    --apple-id "$APPLE_ID" \
    --team-id "$APPLE_TEAM_ID" \
    --password "$APPLE_APP_SPECIFIC_PASSWORD" \
    --wait
    xcrun stapler staple HelixVPN.app

```

Apple Signing Requirements:

- Apple Developer Program membership (\$99/year)
- Developer ID Application certificate for distribution
- Notarization required for macOS 10.15+ (Gatekeeper)
- Hardened Runtime entitlement mandatory
- For iOS: Distribution certificate + provisioning profile

7.3.2 Microsoft: Azure Trusted Signing

"Set environment variables: AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET. These values come from your Azure Active Directory app registration."
[^448^]

Method	Cost	CI-Friendly	SmartScreen
EV Certificate (USB HSM)	\$300-700/year	No (physical token)	Instant trust
EV Cloud Signing	\$300-700/year	Yes	Instant trust
Azure Trusted Signing	Pay-per-use	Yes (native GitHub Actions)	Builds over time
Self-signed	Free	Yes	Warning shown

Windows Code Signing with Azure Trusted Signing

sign-windows:

runs-on: windows-latest

needs: build-desktop

steps:

- name: Download Windows artifact

uses: actions/download-artifact@v4

- name: Install Azure Trusted Signing Client

run: |

dotnet tool install --global Azure.Trusted.Signing.Client

- name: Sign with Azure Trusted Signing

env:

AZURE_TENANT_ID: \${ secrets.AZURE_TENANT_ID }

AZURE_CLIENT_ID: \${ secrets.AZURE_CLIENT_ID }

AZURE_CLIENT_SECRET: \${ secrets.AZURE_CLIENT_SECRET }

TRUSTED_SIGNING_ACCOUNT: \$

{{ secrets.TRUSTED_SIGNING_ACCOUNT }}

CERTIFICATE_PROFILE: \${ secrets.CERTIFICATE_PROFILE }

run: |

SignTool.exe sign /v /debug \

/fd SHA256 \

/tr http://timestamp.acs.microsoft.com \

/td SHA256 \

/dlib Azure.CodeSigning.Dlib.dll \

/dmdf metadata.json \

HelixVPN.msi

7.3.3 Google Play: App Signing

Android Signing

sign-android:

```

runs-on: ubuntu-latest
needs: build-mobile
steps:
  - name: Download Android AAB
    uses: actions/download-artifact@v4

  - name: Sign AAB
    env:
      KEYSTORE_BASE64: ${ secrets.ANDROID_KEYSTORE }
      KEY_PASSWORD: ${ secrets.ANDROID_KEY_PASSWORD }
      KEY_ALIAS: ${ secrets.ANDROID_KEY_ALIAS }
    run: |
      echo "$KEYSTORE_BASE64" | base64 --decode > upload-keystore.jks
      jarsigner -verbose -sigalg SHA256withRSA -digestalg SHA-256 \
        -keystore upload-keystore.jks \
        -storepass "$KEY_PASSWORD" \
        app-release.aab "$KEY_ALIAS"

  - name: Upload to Play Store
    uses: r0adkll/upload-google-play@v1
    with:
      serviceAccountJsonPlainText: $
        { secrets.PLAY_STORE_SERVICE_ACCOUNT }
      packageName: com.helixvpn.app
      releaseFiles: app-release.aab
      track: ${ github.ref == 'refs/heads/main' && 'internal'
        || 'production' }

```

7.3.4 Huawei: AppGallery Signing

HarmonyOS Signing

```

sign-harmonyos:
  runs-on: ubuntu-latest
  needs: build-mobile
  steps:
    - name: Sign HAP
      env:
        HUAWEI_KEYSTORE: ${ secrets.HUAWEI_KEYSTORE }
        HUAWEI_KEY_PASSWORD: ${ secrets.HUAWEI_KEY_PASSWORD }
      run: |
        java -jar hap-sign-tool.jar sign \
          -mode localjks \
          -privatekey "$HUAWEI_KEYSTORE" \
          -signAlg SHA256withECDSA \
          -appCert app.cer \
          -profileDebug profileDebug.p7b \
          -in helix-vpn-unsigned.hap \
          -out helix-vpn-signed.hap

```

7.3.5 Linux: GPG Package Signing (deb/rpm/AppImage)

The §7.1 pipeline diagram lists "Linux: GPG signing" as a stage, but the original draft never specified it concretely (added Revision 2 to close that gap and to keep this section internally consistent — Windows uses an EV/OV-equivalent cloud HSM cert via Azure Trusted Signing §7.3.2, macOS uses Developer ID + notarization §7.3.1, and Linux uses a detached GPG signature, matching how every major Linux repository trust model works).

```
# Linux package signing (.deb / .rpm / .AppImage)
```

```
sign-linux:
```

```
  runs-on: ubuntu-latest
```

```
  needs: build-desktop
```

```
  steps:
```

```
    - name: Download Linux artifacts
```

```
      uses: actions/download-artifact@v4
```

```
    - name: Import release GPG key
```

```
      env:
```

```
        GPG_PRIVATE_KEY: ${ secrets.LINUX_GPG_PRIVATE_KEY }
```

```
        GPG_PASSPHRASE: ${ secrets.LINUX_GPG_PASSPHRASE }
```

```
      run: |
```

```
        echo "$GPG_PRIVATE_KEY" | gpg --batch --import
```

```
        echo "default-cache-ttl 600" >> ~/.gnupg/gpg-agent.conf
```

```
    - name: Sign .deb (dpkg-sig)
```

```
      run: |
```

```
        dpkg-sig --sign builder -k "$
```

```
        ${ secrets.LINUX_GPG_KEY_ID }" \
```

```
        --pgp-options "--batch --pinentry-mode loopback --
```

```
        passphrase ${ secrets.LINUX_GPG_PASSPHRASE }" \
```

```
        helix-vpn_*.deb
```

```
    - name: Sign .rpm (rpmsign)
```

```
      run: |
```

```
        rpmsign --addsign --define "_gpg_name $
```

```
        ${ secrets.LINUX_GPG_KEY_ID }" \
```

```
        helix-vpn-*.rpm
```

```
    - name: Detached-sign AppImage + generate checksums
```

```
      run: |
```

```
        gpg --batch --pinentry-mode loopback \
```

```
        --passphrase "${ secrets.LINUX_GPG_PASSPHRASE }" \
```

```
        --local-user "${ secrets.LINUX_GPG_KEY_ID }" \
```

```
        --detach-sign --armor helix-vpn.AppImage
```

```
        sha256sum helix-vpn.AppImage helix-vpn_*.deb helix-vpn-
```

```
        *.rpm \
```

```
        > SHA256SUMS
```

```
        gpg --batch --pinentry-mode loopback \
```

```
        --passphrase "${ secrets.LINUX_GPG_PASSPHRASE }" \
```

```
--local-user "${{ secrets.LINUX_GPG_KEY_ID }}" \
--clearsign SHA256SUMS
```

Linux Signing Requirements:

- A dedicated release GPG key pair (4096-bit RSA or Ed25519), private key held only in CI secrets + an offline cold-storage backup (§9 data-safety class), never on a developer workstation.
- Public key published on the project website, a public keyserver, and the apt/dnf/zypper repository metadata so package managers verify signatures automatically on install/upgrade.
- .AppImage ships a detached .asc signature plus a GPG-clearsigned SHA256SUMS file (AppImage has no native repository-level signature verification, so the checksum + detached signature is the user-facing verification path).
- Key rotation: annual, or immediately on suspected compromise (composes with §11.4.10 credential-rotation discipline); the previous public key stays published (marked superseded) for a transition window so already-installed repository configs can still verify packages signed before rotation.

7.4 Artifact Generation and Storage

Platform	Artifact Types	Storage	Retention
macOS	.dmg, .pkg (universal)	GitHub Releases + S3	90 days
Windows	.msi, .exe (x86_64)	GitHub Releases + S3	90 days
Linux	.AppImage, .deb, .rpm	GitHub Releases + S3	90 days
iOS	.ipa	App Store Connect	180 days
Android	.aab, .apk	Google Play + S3	90 days
HarmonyOS	.hap	AppGallery + S3	90 days
Web	.wasm, .js	CDN + S3	30 days

7.5 Release Channels

Release Pipeline:

```
[nightly] ----(auto, daily)----> [nightly builds]
```

```

|
v (manual promote)
[beta] -----(48h staging)----> [beta channel]
|
v (5 days confirmation)
[stable] -----(full rollout)---> [production]
|
v (emergency only)
[hotfix] -----(skip stages)----> [forced update]

```

Channel	Trigger	Audience	Criteria to Advance
Nightly	Automated (daily 02:00 UTC)	Internal team	Build succeeds, basic tests pass
Beta	Manual promote from nightly	5% of users	48h stability, crash rate < 0.1%
Stable	Auto-promote from beta	100% of users	5 days beta stability, crash rate < 0.05%
Hotfix	Emergency trigger	100% (forced)	Critical security vulnerability only

7.6 Emergency Security Update Pipeline

Emergency Pipeline Timeline:

```

[Security Alert Received]
|
v (0-1 hour)
[Impact Assessment]
- Severity classification (CVSS)
- Affected platforms and versions
- Exploitability analysis
|
v (1-4 hours for critical)
[Patch Development]
- Dedicated CI pipeline (priority queue)
- Pre-built test environments (no setup)
- Parallel development for all affected platforms
|
v (parallel with dev)
[QA Regression]
- Automated security regression suite
- Smoke tests only (skip full suite)
- Leak test + kill switch verification
|
v (4-8 hours total)
[Fast-track Release]
- Skip staged rollout
- Direct to production channel

```

- Ed25519 signed emergency update
 - |
 - v (immediate)
- [Force Update]
- In-app mandatory update notification
 - Block new connections until updated
 - Consider key rotation if credentials compromised
-

8. Testing Strategy

8.0 Anti-Bluff Testing Doctrine

This section states, in mvp2's own words, the same testing philosophy already in force for the Phase 0/1 control plane — docs/research/mvp/final/10-testing-acceptance-and-qa.md. That document's own one-sentence rule, quoted verbatim: *"A green test suite is not proof the feature works. It only counts as proof if a real artifact — a packet capture, a throughput number, a database rowset, a screen recording — was captured while the feature ran and shows the user-visible outcome actually happened."* This document previously stated no equivalent principle anywhere — an omission named by docs/research/CROSS_CUTTING_GAP_ANALYSIS.md §2.2 — and is corrected here rather than left to silently diverge from the control-plane doctrine.

The mvp2 anti-bluff bar, stated for the 8 client platforms:

1. **A test claim is only true if a real artifact was captured while the client actually ran.** "Connect works," "kill switch trips in under 100ms," "no DNS leak" are only proven by a real pcap (§8.3), a real iperf3/ping number (§8.4), a real WebDriver/XCUITest/Espresso/Playwright session driving the actually-built app (§8.5), or a real before/after policy-enforcement diff (§10.5) — never a green exit code or an assertion that merely returned true.
2. **Real UI, driven end-to-end, never mocked above the unit layer.** Every §8.5 UI test drives the actually-built application through its real UI — tauri-driver against the real webview, XCUITest/Espresso against a real device or simulator, Playwright against the real built extension/PWA in a real browser profile — never a stubbed status object or a fake chrome.* shim. §8.1's Rust unit tests are the **only** layer where a mock/stub/fake is permitted; §8.2 integration tests boot the real platform VPN primitive (NEPacketTunnelProvider, WFP, VpnService) and the real Rust core, never a mocked core.
3. **Real network conditions, not a synthetic shortcut.** The kill-switch and DNS-leak assertions in §8.3 capture a pcap and inspect it for zero non-VPN egress and zero :53 traffic during the gap — a timer-only elapsed-time assertion is necessary but

not sufficient; the pcap is the artifact that actually proves the claim, matching the evidence bar 10-testing-acceptance-and-qa.md §3 already requires for the identical claim on the control-plane side.

4. **Self-validated analyzers.** Any custom leak/OCR/traffic-pattern analyzer this document's harnesses rely on ships a golden-good + golden-bad fixture pair so the analyzer itself cannot rubber-stamp a broken feature as passing — the same discipline 10-testing-acceptance-and-qa.md §3.3 requires project-wide.
5. **Mocks are confined to §8.1 (Rust unit tests) only.** This is the explicit boundary statement this document previously left implicit: a future contributor who mocks the VPN core, the platform VPN API, or the network at the §8.2 integration layer or above has violated this document's own testing policy, not merely taken a shortcut.

Coverage-ledger note. This document currently expresses coverage only as static percentage targets (§8.1's Coverage Targets table) — a materially weaker guarantee than the git-tracked, schema-CHECK-constrained coverage ledger 10-testing-acceptance-and-qa.md §6 already implements for the control plane (a constraint that makes a bluffed "verified" row structurally impossible: 100% line coverage of an assertion-free function is still "100%," which the ledger's evidence-state model rejects). Extending that same ledger with client-platform feature_id rows (e.g. F-CONNECT-FLOW-DESKTOP, F-KILL-SWITCH-MOBILE) — rather than mvp2/ inventing a second, unlinked coverage-tracking mechanism — is tracked follow-up work per docs/research/CROSS_CUTTING_GAP_ANALYSIS.md §2.3 item 6.

8.1 Unit Tests: Rust Core

```
// Unit Test Structure for Rust Core
#[cfg(test)]
mod tests {
    use super::*;

    // 1. Cryptographic Operation Tests
    #[test]
    fn test_x25519_key_exchange() {
        let alice = X25519KeyPair::generate();
        let bob = X25519KeyPair::generate();

        let alice_shared = alice.diffie_hellman(bob.public_key());
        let bob_shared = bob.diffie_hellman(alice.public_key());

        assert_eq!(alice_shared.as_bytes(),
                   bob_shared.as_bytes());
    }
}
```

```

#[test]
fn test_chacha20_poly1305_roundtrip() {
    let key = ChaCha20Poly1305Key::generate();
    let plaintext = b"Hello, Helix VPN!";
    let nonce = Nonce::generate();
    let aad = b"additional authenticated data";

    let ciphertext = key.encrypt(plaintext, &nonce,
                                aad).unwrap();
    let decrypted = key.decrypt(&ciphertext, &nonce,
                                aad).unwrap();

    assert_eq!(plaintext.to_vec(), decrypted);
}

#[test]
fn test_wireguard_handshake() {
    let mut initiator = WireGuardState::new(initiator_keys);
    let mut responder = WireGuardState::new(responder_keys);

    let handshake_init =
        initiator.create_handshake_initiation();
    let handshake_resp =
        responder.process_handshake_initiation(&handshake_init);
    initiator.process_handshake_response(&handshake_resp);

    assert!(initiator.is_handshake_complete());
    assert!(responder.is_handshake_complete());
    assert_eq!(initiator.send_key(), responder.receive_key());
    assert_eq!(initiator.receive_key(), responder.send_key());
}

#[test]
fn test_packet_encoding() {
    let packet = WireGuardDataPacket::new()
        .with_receiver_index(12345)
        .with_counter(0)
        .with_encrypted_payload(vec![1, 2, 3, 4]);

    let encoded = packet.encode();
    let decoded =
        WireGuardDataPacket::decode(&encoded).unwrap();

    assert_eq!(packet.receiver_index(),
                decoded.receiver_index());
    assert_eq!(packet.counter(), decoded.counter());
}

#[test]
fn test_hkdf_key_derivation() {
    let ikm = b"input key material";
    let salt = b"salt value";
    let info = b"info string";

```

```

    let okm = hkdf_sha256(ikm, salt, info, 32);

    // Verify against known test vector
    assert_eq!(okm, hex!("expected_output_here"));
}

#[test]
fn test_memory_zeroization() {
    let mut secret = SecretKey::generate();
    secret.zeroize();

    // Verify all bytes are zeroed
    assert!(secret.as_bytes().iter().all(|&b| b == 0));
}
}

```

Coverage Targets:

Module	Target Coverage	Critical Paths (must be 100%)
Crypto operations	>90%	All cipher roundtrips, key derivation
WireGuard protocol	>85%	Handshake, packet encoding, rekey
Shadowsocks protocol	>80%	AEAD-2022 encryption, TCP/UDP relay
Kill switch logic	>90%	All platform implementations
DNS handling	>85%	All resolver paths, leak prevention
Key management	>90%	Storage, rotation, zeroization
Configuration	>75%	Parsing, validation, migration
UI state management	>70%	Connection state machine

8.2 Integration Tests: Per-Platform Connection Tests

Test Category	Test Cases	Tools
Connection establishment	Successful connect, auth failure, timeout, wrong credentials	Custom test harness, curl
Reconnection	Network change, sleep/wake, airplane	Platform-specific frameworks

Test Category	Test Cases	Tools
	mode, app backgrounding	
Kill switch	Disconnect mid-transfer, verify no leaks, response time	tcpdump, Wireshark, custom scripts
DNS handling	DNS leak test, custom DNS, DoH/DoT/DoQ resolution	dnsleaktest.com, dig, tcpdump
IPv6 handling	IPv6 disabled, IPv6 tunneled, IPv6 leak detection	ping6, tcpdump, test-ipv6.com
Split tunneling	Included routes, excluded routes, per-app routing	route/netstat, curl with bind
Protocol fallback	Primary blocked, fallback to Shadowsocks, MASQUE	Custom DPI simulation
Performance	Throughput, latency, CPU usage, memory usage	iperf3, ping, htop, custom scripts

8.3 Leak Tests

```
#!/bin/bash
# Automated Leak Test Suite

# 1. DNS Leak Test
echo "=== DNS Leak Test ==="
connect_vpn
for dns_server in $(dig +short whoami.dnsleaktest.com); do
    if [[ "$dns_server" != "$EXPECTED_DNS" ]]; then
        echo "FAIL: DNS leak detected: $dns_server"
        exit 1
    fi
done
echo "PASS: No DNS leak"

# 2. WebRTC Leak Test
echo "=== WebRTC Leak Test ==="
# Use headless browser to check WebRTC IPs
WEBRTC_IPS=$(node -e "
    const puppeteer = require('puppeteer');
    (async () => {
        const browser = await puppeteer.launch();
        const page = await browser.newPage();
        await page.goto('https://browserleaks.com/webrtc');
    })();
")
```

```

        // Extract IP addresses from page
        const ips = await page.evaluate(() => {
            return document.querySelectorAll('.ip-address').map(e
=> e.textContent);
        });
        console.log(ips.join('\n'));
        await browser.close();
    })();
")
echo "WebRTC IPs: $WEBRTC_IPS"

# 3. IPv6 Leak Test
echo "=== IPv6 Leak Test ==="
if ping6 -c 1 test-ipv6.com 2>/dev/null; then
    echo "FAIL: IPv6 traffic leaking outside VPN"
    exit 1
else
    echo "PASS: IPv6 blocked"
fi

# 4. Kill Switch Response Time Test
echo "=== Kill Switch Response Time ==="
# Measure time between disconnect and traffic block
disconnect_vpn &
START_TIME=$(date +%s%N)
# Attempt connection that should be blocked
while curl --max-time 1 http://example.com 2>/dev/null; do
    : # Keep trying

done
END_TIME=$(date +%s%N)
RESPONSE_TIME=$(( (END_TIME - START_TIME) / 1000000 )) # Convert
to ms
echo "Kill switch response: ${RESPONSE_TIME}ms"
if [ "$RESPONSE_TIME" -gt 100 ]; then
    echo "FAIL: Kill switch too slow (>100ms)"
    exit 1
fi
echo "PASS: Kill switch < 100ms"

```

8.4 Performance Tests

```

// Rust Performance Benchmarks
#[cfg(test)]
mod benchmarks {
    use test::Bencher;

    #[bench]
    fn bench_wireguard_encrypt(b: &mut Bencher) {
        let key = SessionKey::generate();
        let packet = vec![0u8; 1400]; // Typical MTU-sized packet
    }
}

```

```

        let nonce = Nonce::generate();

        b.iter(|| {
            key.encrypt(&packet, &nonce, &[])
        });
    }

#[bench]
fn bench_handshake_initiation(b: &mut Bencher) {
    let state = WireGuardState::new(keys);

    b.iter(|| {
        state.create_handshake_initiation()
    });
}

#[bench]
fn bench_packet_throughput(b: &mut Bencher) {
    let mut tunnel = create_test_tunnel();
    let packets: Vec<Vec<u8>> = (0..1000)
        .map(|_| random_packet(1400))
        .collect();

    b.iter(|| {
        for packet in &packets {
            tunnel.send(packet).unwrap();
        }
    });
}
}

```

Performance Test Matrix:

Test	Desktop Target	Mobile Target	Frequency
Throughput (iperf3)	> 500 Mbps	> 100 Mbps	Per release
Latency (ping)	< 1ms overhead	< 2ms overhead	Per release
Connection time	< 100ms	< 200ms	Per release
Memory footprint	< 100MB	< 50MB	Per release
CPU at max throughput	< 25%	< 40%	Per release
Battery (1 hour idle)	N/A	< 3%	Per release

8.5 UI Tests

Desktop (Tauri):

```
// Tauri UI tests using tauri-driver
#[test]
fn test_connection_flow() {
    let driver = WebDriver::new("http://localhost:4444",
        capabilities).unwrap();

    // Open app
    driver.get("tauri://localhost").unwrap();

    // Click connect button
    let connect_btn = driver.find_element(By::Id("connect-
        btn")).unwrap();
    connect_btn.click().unwrap();

    // Wait for connected state
    driver.wait_for_element(By::ClassName("connected")).unwrap();

    // Verify status shows connected
    let status = driver.find_element(By::Id("status-
        text")).unwrap();
    assert!(status.text().unwrap().contains("Connected"));

    // Click disconnect
    let disconnect_btn = driver.find_element(By::Id("disconnect-
        btn")).unwrap();
    disconnect_btn.click().unwrap();

    // Verify disconnected
    driver.wait_for_element(By::ClassName("disconnected")).unwrap();
}
```

Mobile:

- iOS: XCTest + XCUITest for UI automation
- Android: Espresso + UI Automator
- Test: Connect button, server selection, protocol toggle, settings changes

8.6 Security Tests: Penetration Testing Checklist

#	Test	Method	Expected Result
1	DNS leak test	Multiple DNS leak websites	No real DNS servers visible
2	WebRTC leak test	browserleaks.com/webrtc	Only VPN IP visible
3	IPv6 leak test	test-ipv6.com	IPv6 unreachable

#	Test	Method	Expected Result
4	Kill switch test	tcpdump during disconnect	No plaintext traffic > 100ms
5	Key extraction	Memory dump analysis	Keys zeroized, not in swap
6	Binary tampering	Modify signed binary	Signature verification fails
7	Downgrade attack	Serve older version	Version check rejects
8	Replay attack	Replay WireGuard handshake	Replay detection triggers
9	MITM certificate	Use invalid CA cert	Certificate pinning rejects
10	Traffic analysis	Statistical packet analysis	No identifiable patterns (with DAITA)
11	Side-channel timing	Constant-time verification	No timing differences
12	Update signature	Modified update package	Signature verification fails
13	Credential storage	Keychain/Keystore extraction	Hardware-backed, non-exportable
14	Jailbreak/root	Compromised device	Detection + warning/block

8.7 CI Test Matrix

Cadence note (Revision 3): "Daily" / "Weekly" / "Per release" below describe **local gate cadence** (pre-commit / pre-push / make qa triggers run on a developer or release machine), not a remote CI service — per constitution §11.4.156, all CI/CD automation is disabled and there is no active `.github/workflows/*.yml` or `.gitlab-ci.yml` in this project, matching the local-only test-pyramid topology docs/research/mvp/final/10-testing-acceptance-and-qa.md §4 already implements for the control plane (docs/research/CROSS_CUTTING_GAP_ANALYSIS.md §2.2 item 3).

Platform	Unit Tests	Integration	E2E	Security Tests	Performance
Linux	cargo test	Docker-based	Tauri WebDriver	Daily	Per release
macOS	cargo test	NEPacketTunnel	XCUITest	Daily	Per release
Windows	cargo test	WFP integration	WinAppDriver	Daily	Per release

Platform	Unit Tests	Integration	E2E	Security Tests	Performance
iOS	cargo test (sim)	Device farm	XCTest	Weekly	Per release
Android	cargo test	Emulator	Espresso	Weekly	Per release
Web	wasm-pack test	Browser tests	Playwright	Daily	Per release

9. Compliance & Certification

9.1 SOC 2 Type II Preparation

SOC 2 Trust Service Criteria Applicable to VPN:

Criteria	VPN-Specific Controls	Implementation
Security	Encryption standards, access controls, vulnerability management	AES-256-GCM, ChaCha20-Poly1305, hardware-backed keys, automated patching
Availability	Uptime monitoring, redundancy, failover mechanisms	Multi-server deployment, health checks, automatic failover
Processing Integrity	Accurate packet routing, no data corruption, protocol compliance	Protocol test suite, checksum verification, end-to-end testing
Confidentiality	Encryption in transit, key management, secure protocols	TLS 1.3, WireGuard, secure key storage, PFS
Privacy	No-logs policy, minimal data collection, user consent	No traffic logging, data minimization, clear privacy policy

SOC 2 Implementation Requirements:

- Documented security policies and procedures
- Change management with approval workflows
- Incident response plan with defined SLAs
- Regular vulnerability scanning and penetration testing
- Access logging (for infrastructure, not user traffic)
- Employee background checks and security training
- Third-party risk assessment for vendors

9.2 ISO 27001 Mapping

ISO 27001:2022 Control	VPN Implementation	Status
A.5.1 Information security policies	Documented security policy, review cycle	Required
A.6.1 Organization of information security	Security roles and responsibilities defined	Required
A.8.1 User endpoint devices	Secure client distribution, device posture check	Required
A.8.2 Information deletion	Secure key wipe on uninstall	Required
A.8.6 Capacity management	Server capacity monitoring, auto-scaling	Required
A.8.9 Configuration management	Version-controlled configs, change tracking	Required
A.10.1 Cryptographic controls	AES-256-GCM, ChaCha20-Poly1305, X25519	Required
A.12.1 Network security management	Firewall rules, network segmentation, kill switch	Required
A.12.4 Logging and monitoring	Infrastructure monitoring (not user activity)	Required
A.13.1 Network security	TLS 1.3, certificate pinning, mutual auth	Required
A.14.2 Secure development	SAST/DAST in CI, code review requirements	Required
A.16.1 Management of information security incidents	Incident response plan, breach notification	Required

9.3 GDPR Compliance

"Regulations like GDPR and CCPA influence how businesses deploy and manage VPNs to comply with data protection and privacy laws." [^442^]

GDPR Principle	VPN Implementation
Lawful basis	Legitimate interest (security) or explicit consent
Data minimization	Collect only: account ID, subscription status, app version
Purpose limitation	Data used only for VPN service delivery
Storage limitation	Delete account data within 30 days of account closure
Security	Encryption, integrity, confidentiality measures
Accountability	Documented data processing activities
No-logs policy	No traffic content logs, no connection metadata logs

Data Processing Inventory:

Data Category	Collected	Stored	Retention	Purpose
Account email	Yes	Yes	Account lifetime	Authentication
Payment info	No	No	N/A	Handled by Stripe/PayPal
Traffic content	No	No	N/A	Never collected
Connection timestamps	No	No	N/A	Never collected
Bandwidth usage	Yes (aggregate)	Yes	30 days	Capacity planning
App crash logs	Yes (anonymized)	Yes	90 days	Quality improvement
Device type	Yes	Yes	Account lifetime	Support

9.4 No-Logs Policy Verification Approach

"In rigorous lab testing across 17 major providers (2023-2024), only Proton VPN, Mullvad, and IVPN consistently prevented DNS, WebRTC, and IPv6 leaks during active transfers. All three passed independent audits

(Cure53, Syss, Assured) confirming no persistent logs of connection timestamps, IP addresses, or transferred file metadata." [^398^]

Verification Framework:

Layer	Method	Frequency	Auditor
Infrastructure audit	Verify no logging servers, storage, or configuration	Annual	Third-party security firm
Code audit	Verify client and server don't create logs	Annual	Security research firm (Cure53, Syss)
Network audit	Verify no telemetry or analytics traffic	Annual	Independent network analysis
Runtime verification	Monitor all outbound connections from VPN servers	Continuous	Automated monitoring
Jurisdiction	Operate in privacy-friendly jurisdictions	Ongoing	Legal review

9.5 Third-Party Security Audit Schedule

Audit Type	Frequency	Scope	Expected Cost
Source code audit	Annual	Rust core, crypto implementation, kill switch	\$50K-150K
Penetration test	Bi-annual	Client apps, infrastructure, APIs	\$30K-80K
No-logs verification	Annual	Infrastructure, network traffic analysis	\$40K-100K
Mobile app audit	Annual	iOS/Android apps, keychain/keystore usage	\$30K-60K
Compliance audit	Annual (SOC 2)	All controls, documentation	\$50K-200K

10. Enterprise Hardening & Production Readiness

(Added Revision 2. This section fulfills the cross-reference made by MVP2_ARCHITECTURE.md §10.1/§10.3 — "Full mechanism detail ... is specified in MVP2_SECURITY_PERFORMANCE.md §10" — and by MVP2_SHARE D_CORE.md §5.5 — "the release-process detail (signing, attestation storage, canary gating) lives in MVP2_SECURITY_PERFORMANCE.md §10." It does not duplicate the architectural contract already stated there (which platform owns which managed-config transport, which build produces which SBOM); it specifies the concrete mechanism each contract runs on.)

10.1 Staged / Canary Rollout Across All 8 Platforms

Canary ring definition (applies uniformly; per-platform mechanism follows in the table below):

Ring	Audience	Minimum Duration	Advance Criteria
Internal / Dogfood	Employees + opted-in volunteer testers (~50-200 devices)	3 business days	Build succeeds, §8 test matrix green, no P0/P1 crash
Canary 1%	~1% of active install base per platform	24h	Rollout Health Monitor (§10.2) reports no regression
Canary 10%	~10%	24h	Rollout Health Monitor reports no regression
Canary 50%	~50%	24h	Rollout Health Monitor reports no regression
GA 100%	100%	n/a — monitoring continues indefinitely	Reproducible-build gate (§10.7) already confirmed for this tag

This refines the existing §7.5 Beta/Stable channel model into discrete, independently-monitorable percentage rings rather than a single 5%→100% step, so a regression is caught (and halted) at 1% of the install base instead of at 5%.

Per-platform rollout mechanism — every platform's own distribution channel is used as-is; none is reinvented:

Platform	Distribution Channel	Native Staged Rollout?	Mechanism / Compensating Control
macOS / Windows / Linux (Tauri)	Self-hosted tauri-plugin-updater endpoint + .deb/.rpm/AppImage repos	Approximated (self-hosted, so we control it)	Update-server buckets each device by <code>sha256(anonymous_install_id) % 100</code> ; only devices whose bucket falls within the currently-open percentage range receive the new-version manifest (§10.1.1 below)
Android	Google Play Console	Yes (native)	Play Console "staged rollout" — configurable percentage, pausable/resumable directly from the console; MVP2 policy sets it to the ring percentages above
iOS	App Store Connect	Yes (native, fixed schedule)	"Phased Release for Automatic Updates" — Apple's fixed 7-day schedule (Day 1: 1%, Day 2: 2%, Day 3: 5%, Day 4: 10%, Day 5: 20%, Day 6: 50%, Day 7: 100%); can be paused but not re-percentaged — MVP2's Canary 1%/10%/50% rings map approximately onto Apple's Day 1/4/6 checkpoints
HarmonyOS	Huawei AppGallery Connect	Yes (native)	AppGallery Connect "Phased Rollout" (gray release) — configurable percentage from the console, matching the ring table
Aurora OS	Aurora OS Store / OpenRepos / enterprise MDM push	No — honestly, neither channel offers vendor-side percentage rollout	Compensating control: extended internal dogfood window (minimum 10 business days, vs. the 3-day default) plus a mandatory enterprise-pilot-fleet sign-off gate before any wider push, since Aurora OS deployments are already typically enterprise-MDM-managed device fleets (§10.5) rather than open

Platform	Distribution Channel	Native Staged Rollout?	Mechanism / Compensating Control
Web — Browser Extension (MV3)	Chrome Web Store / Firefox AMO / Edge Add-ons / Safari Extensions	No — extension stores auto-update all installed users to the latest published version (no vendor percentage-rollout API exists for any of the four stores)	consumer installs — the pilot fleet functions as the missing canary ring
			Compensating control: feature-flag kill switch (§10.1.2) — ship the extension build to 100% but gate any new or risky code path behind a remote flag queried from the MVP1 Utils Service at startup, so a bad rollout is mitigated by disabling behavior remotely rather than by a slower store rollout
Web — PWA	Self-hosted (service worker update event)	Approximated	Same bucketed-manifest technique as desktop (§10.1.1), applied to the service worker's update-check response

10.1.1 Bucketed Update-Manifest Rollout (Tauri / PWA)

```
// Update-server response logic (pseudo-code – not client-side config)
// GET /updates/{platform}/{arch}/latest.json?install_id=<anon-id>
{
  "policy": {
    "current_ring": "canary_10",
    "ring_bucket_ranges": {
      "canary_1": [0, 1),
      "canary_10": [0, 10),
      "canary_50": [0, 50),
      "ga_100": [0, 100)
    }
  },
  "rule": "bucket = sha256(install_id) % 100; serve new manifest only if bucket falls inside ring_bucket_ranges[current_ring]; otherwise return the previous stable manifest unchanged"
}
```

install_id is a random, non-account-linked UUID generated at first launch (never derived from hardware identifiers — composes with §9.3 GDPR data minimization). The same bucket assignment

is reused as each ring widens (a device that was in canary_1 stays included as the range grows to canary_10), so a device is never "rolled back" to an older manifest by a bucket recompute, only ever advanced.

10.1.2 Feature-Flag Kill Switch (Utils Service)

For the two channels with no vendor-side staged rollout (browser extensions, and Aurora OS as a compensating measure), new or risky behavior added in a release ships behind a remotely-toggleable flag:

```
# Utils Service feature-flag check (client startup, all platforms
  - not just
# the two above; the same mechanism doubles as the §10.2 rollback
  lever)
GET /v1/flags?client_version=2.4.0&platform=web-extension
# Response:
{
  "flags": {
    "new_obfuscation_negotiation_path": false,
    "adaptive_mtu_v2": true
  },
  "policy_version": 14
}
```

A flag flip takes effect on the client's next flag poll (default interval: 5 minutes while connected, on-launch otherwise) — no new extension publish, no app-store review cycle, no user action required.

10.2 Rollout Health Monitor & Automatic Rollback

The monitor watches two signals per active ring, both sourced from existing MVP1 surfaces (no new backend service invented): **crash-rate delta**, from the crash-reporting pipeline (§10.3), and **error-rate delta**, from Client API telemetry (connection failures, handshake failures, protocol negotiation failures reported by the client).

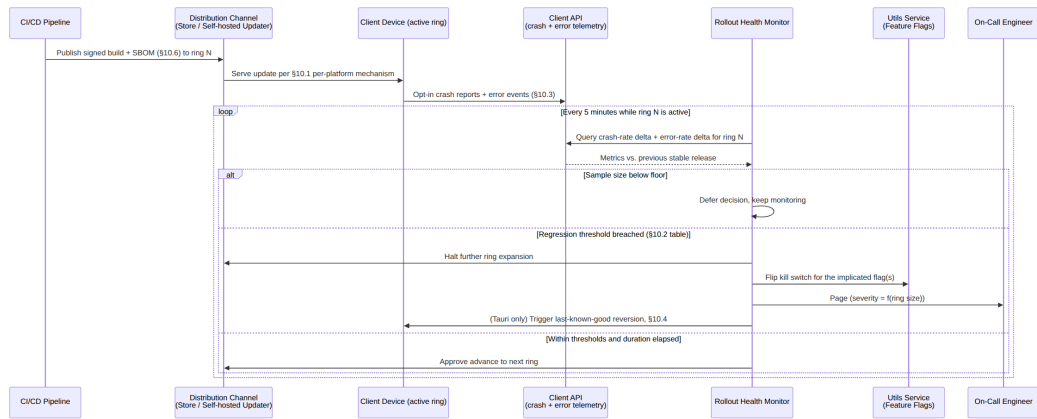
Ring	Crash-Rate Regression Threshold (vs. previous stable)	Error-Rate Regression Threshold	Action on Breach
Canary 1%	> +0.05 percentage points absolute, OR > 2× relative	> +1.0 percentage points absolute	Auto-halt ring expansion + auto- flip feature-flag kill switch (§10.1.2) + page on-call

Ring	Crash-Rate Regression Threshold (vs. previous stable)	Error-Rate Regression Threshold	Action on Breach
Canary 10%	> +0.03pp absolute, OR > 1.5× relative	> +0.5pp absolute	Same
Canary 50%	> +0.02pp absolute, OR > 1.3× relative	> +0.3pp absolute	Same
GA 100%	> +0.02pp absolute, sustained 1 hour	> +0.3pp absolute, sustained 1 hour	No further ring to halt; escalates directly to the §7.6 Emergency Security Update Pipeline if severity warrants

Sample-size floor: a ring's metrics are not evaluated against these thresholds until it has accumulated a statistically meaningful sample (minimum 200 sessions or 4 hours elapsed, whichever is later) — evaluating too early on a tiny sample is a false-positive risk, not a safety margin.

"Automatic rollback" is intentionally scoped honestly per channel — it is **never** a forced uninstall of an already-updated app:

- **Halt** (all channels): stop advancing the ring — Play Console / App Store Connect / AppGallery Connect rollout is paused via their own API; the bucketed-manifest server (§10.1.1) freezes `current_ring` at its last value.
- **Flag flip** (all channels): the feature-flag kill switch (§10.1.2) disables the specific risky behavior for already-updated devices within minutes, without a new release.
- **Reissue previous-version installer** (Tauri desktop only, where the client controls its own update mechanism): the client-side rollback described in §10.4 actively reverts an already-updated device back to the last-known-good signed installer.
- **Alert:** PagerDuty (or equivalent) page to on-call, severity scaled to ring size (a Canary 1% breach is far cheaper to page loudly on than a GA 100% breach that's already been live for days).



This complements, rather than duplicates, the higher-level rollout diagram in MVP2_ARCHITECTURE.md §10.1 (flowchart LR across the whole pipeline); this diagram is the detailed polling/decision loop inside that flowchart's MONITOR subgraph.

10.3 Crash Reporting & Telemetry Pipeline (Privacy-First)

Every platform's own "Enterprise Hardening" section (being hardened in parallel) plugs its native crash SDK into this one pipeline rather than standing up per-platform infrastructure.

Architecture:

Stage	Detail
Ingestion endpoint	Dedicated Client API routes <code>POST /v1/telemetry/crash</code> and <code>POST /v1/telemetry/error</code> , authenticated by the same non-account-linked <code>install_id</code> used for rollout bucketing (§10.1.1) — never the user's account token, so a crash report cannot be joined back to an identity server-side
PII scrubbing	Applied at the ingestion edge, before any storage write: source IP address is never persisted (this is a VPN client — logging source IPs would itself violate the §9.4 no-logs policy); free-text fields (user-supplied bug-report comments) pass through a scrubber regex stripping emails, IPv4/IPv6 literals, and bearer tokens; file paths in stack traces are normalized to strip OS username segments (<code>/Users/<name>/</code> → <code>/Users/<redacted>/</code>); device identifiers are salted-hashed with a rotating (30-day) salt, never stored raw
Symbolication pipeline	See CI snippet below — dSYM (Apple), ProGuard/R8 mapping.txt (Android), <code>dump_syms/</code> Breakpad .sym (Rust core, all platforms), and

Stage	Detail
	Windows PDB are all uploaded to a private symbol server in CI, keyed by exact build + version, so crash payloads never need to embed unstripped symbols on-device
Retention	Raw (symbolicated) crash payloads: 90 days — matches the existing §9.3 GDPR Data Processing Inventory row ("App crash logs Yes (anonymized) Yes 90 days Quality improvement"), not a new retention rule. Aggregated crash-rate / error-rate metrics (counts only, no payload, no stack trace): retained indefinitely for trend analysis, consistent with data minimization since no PII exists in an aggregate count
Consent gate	First-run consent dialog states exactly what §9.3's table already discloses (anonymized crash logs, 90-day retention, quality-improvement purpose). Consent state is stored locally and mirrored to the MVP1 Utils Service as feature-flag key telemetry_crash_reporting_enabled; when a user opts out, the crash SDK runs in local-only mode (writes to a local log file for user-initiated "Send diagnostic report" actions only — nothing is transmitted automatically)

```
# Symbolication upload, run in CI immediately after each signed
build
upload-symbols:
  needs: [sign-macos, sign-windows, sign-android, sign-harmonyos,
    build-desktop]
  runs-on: ubuntu-latest
  steps:
    - name: Upload Apple dSYMs
      run: symbol-server upload --platform ios --version $
        {{ github.ref_name }} HelixVPN.dSYM.zip

    - name: Upload Android ProGuard/R8 mapping
      run: symbol-server upload --platform android --version-code
        ${ env.VERSION_CODE } mapping.txt

    - name: Upload Rust core Breakpad symbols
      run: |
        dump_syms target/release/libhelix_core.so >
        libhelix_core.sym
        symbol-server upload --platform rust-core --version $
        {{ github.ref_name }} libhelix_core.sym
```

```
- name: Upload Windows PDB
  run: symbol-server upload --platform windows --version $
      {{ github.ref_name }} HelixVPN.pdb
```

Cross-reference: this pipeline's data handling is bound by, and must not diverge from, the constraints already documented in §9.3 GDPR Compliance and §9.4 No-Logs Policy Verification Approach — §10.3 is the mechanism, §9.3/§9.4 remain the compliance authority.

10.4 Client-Side Update Rollback (Tauri Desktop)

For macOS/Windows/Linux, where the client itself (not an app store) owns the update mechanism via tauri-plugin-updater, the client can perform a genuine reversion — not merely a halted rollout — for an already-updated device.

```
// tauri.conf.json (relevant excerpt)
{
  "plugins": {
    "updater": {
      "active": true,
      "endpoints": [
        "https://updates.helixvpn.example/{{target}}/{{arch}}/
{{current_version}}"
      ],
      "dialog": true,
      "pubkey": "<Ed25519 public key, matches §2 update-mechanism
signing key>",
      "windows": { "installMode": "passive" }
    }
  }
}
```

- **Keep N previous signed installers cached.** On every successful update, the client retains the previous $N = 3$ signed installers (current, current-1, current-2) under `<app-data-dir>/previous-versions/`, each with its original Ed25519 signature and SHA-256 checksum intact — never re-derived, so a cached installer's signature always verifies against the same public key the live updater trusts.
- **Local crash-count watchdog.** The Rust core increments a small per-boot counter (`<app-data-dir>/update-health.json`) whenever the previous process instance exited abnormally (crash, not a clean shutdown) within the first 48 hours after an update. This is a local, device-side signal — independent of, and faster-reacting than, the aggregate §10.2 Rollout Health Monitor, which needs many devices' aggregate data before it can act.
- **Auto-revert trigger.** If the local crash count reaches ≥ 3 within a rolling 24-hour window **and** the currently-installed version was installed within the last 48 hours, the client

automatically launches the cached current-1 installer in silent/passive mode, notifies the user ("Helix VPN reverted to the previous stable version after detecting repeated crashes"), and reports the auto-rollback event (opt-in telemetry, §10.3) so the Rollout Health Monitor sees the same signal in aggregate across devices, even though this trigger acted locally and immediately rather than waiting for the aggregate view.

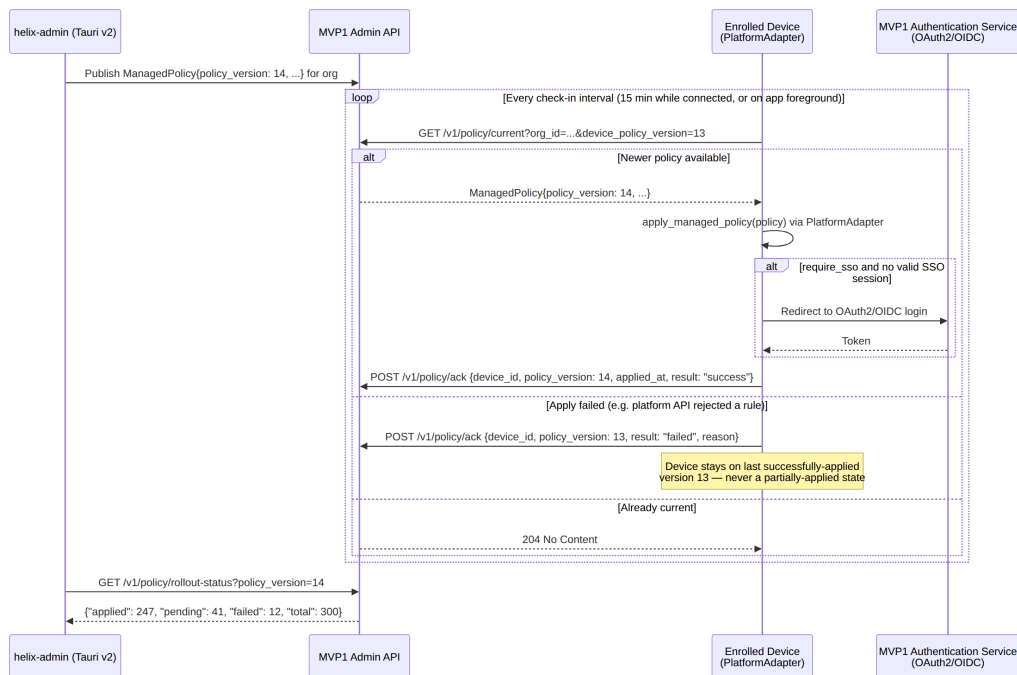
- **Fail-safe bound.** If no current-1 installer is cached (e.g., first install), the watchdog cannot revert and instead surfaces the crash count to the user with a manual "reinstall previous version from helixvpn.example" link — it never attempts to synthesize or download an unverified fallback binary.

10.5 Enterprise Policy Push Architecture

End-to-end flow for helix-admin (Tauri v2 fleet-management dashboard, MVP2_ARCHITECTURE.md §2.2.6) pushing org policy to enrolled devices via the MVP1 Admin API, and each platform's PlatformAdapter::apply_managed_policy (MVP2_ARCHITECTURE.md §5.5) applying it.

```
// ManagedPolicy – pushed by helix-admin, applied by every
// platform adapter
pub struct ManagedPolicy {
    pub policy_version: u64,           // monotonic; devices never
    // apply an                        // older version than one
    // already applied
    pub allowed_protocols: Vec<ProtocolId>,
    pub forced_kill_switch: bool,
    pub forced_split_tunnel_rules: Vec<SplitTunnelRule>,
    pub forced_dns_servers: Vec<IpAddr>,
    pub require_sso: bool,             // bridges to the MVP1
    // Authentication                  // Service's existing OAuth2/
    // OIDC support
}
```

Device check-in / acknowledgment model (so the dashboard can show "247/300 devices applied policy v14"):



Failure handling: a device that fails to apply retries at the next check-in with exponential backoff (composes with §11.4.147 crash-respawn discipline at the fleet level, not just the agent level); it never half-applies a policy — `apply_managed_policy` is treated as atomic per device (all rules applied, or the previous policy version remains active and the failure reason is surfaced in the ack for the helix-admin operator to triage).

10.6 Supply-Chain Security — Release-Process Gates

MVP2_SHARED_CORE.md §5.5 ("Supply-Chain & Build Integrity") owns SBOM *generation* (cargo-cyclonedx for the Rust core, syft for Tauri/browser bundles, cyclonedx-flutter for mobile) and states that "the release-process detail (signing, attestation storage, canary gating) lives in MVP2_SECURITY_PERFORMANCE.md §10" — this is that detail.

- **Publication:** every generated SBOM is attached to the corresponding GitHub Release alongside the signed artifact and its SHA256SUMS checksum file (the same checksum file produced in §7.3.5 for Linux, and analogous per-artifact checksums for the other platforms) — an auditor or enterprise customer downloads one signed artifact and gets its checksum, its GPG/code-signing signature, and its SBOM as a matched set.
- **Release-to-release diff gate:** composes with, and does not replace, the cargo audit / cargo deny check / cargo vet vulnerability and license gates already in the §7.1/§7.2 pipeline — those catch *known vulnerabilities and disallowed licenses*;

this gate catches *unreviewed new dependencies*, a different failure mode (supply-chain injection).

```
# Runs after SBOM generation (MVP2_SHARED_CORE.md §5.5), before
  promoting
# past the Internal/Dogfood ring
sbom-diff-gate:
  needs: [generate-sboms]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Fetch previous release's SBOM
      run: gh release download "$(gh release list -L 1 --exclude-
        drafts --json tagName -q '.[0].tagName')" --pattern
        "*.cdx.json" --dir previous-sbom/

    - name: Diff CycloneDX component lists
      run: |
        jq -r '.components[].purl' previous-sbom/*.cdx.json | sort
        -u > previous-deps.txt
        jq -r '.components[].purl' current-sbom/*.cdx.json | sort
        -u > current-deps.txt
        comm -13 previous-deps.txt current-deps.txt > new-deps.txt
        cat new-deps.txt

    - name: Fail on unreviewed new dependency
      run: |
        # .helix/sbom-reviewed-deps.txt is checked into the repo;
        a new
        # dependency is added to it only via a PR that a reviewer
        approved
        # specifically for that addition (composes with §11.4.142
        universal
        # code-review mandate)
        comm -23 new-deps.txt <(sort .helix/sbom-reviewed-
        deps.txt) > unreviewed-deps.txt
        if [ -s unreviewed-deps.txt ]; then
          echo "::error::Unreviewed new dependency introduced
          since last release:"
          cat unreviewed-deps.txt
          exit 1
        fi
```

10.7 Reproducible Build Verification as a Release Gate

MVP2_SHARED_CORE.md §5.5 defines the quarterly reproducibility drill (rebuild the last released tag on a second independent CI runner, diff artifact hashes) as the standing anti-bluff check on the reproducibility *claim*. §10.7 adds the release-time enforcement that claim was missing a mechanism for: **a release is not promoted past the Internal/Dogfood ring (i.e., never**

reaches Canary 1%, §10.1) until reproducibility is confirmed for that specific tag — not merely sampled quarterly, but verified for every release.

*# Runs once per release tag, gates promotion out of the Internal/
Dogfood ring*

verify-reproducibility:

needs: [build-desktop, build-mobile]

runs-on: ubuntu-latest *# deliberately a different runner pool
than the*

*# primary build, per
MVP2_SHARED_CORE.md §5.5*

steps:

- uses: actions/checkout@v4

with:

ref: \${github.ref_name}

- name: Rebuild from pinned toolchain + lockfile

run: |

cargo build --release --locked --target x86_64-unknown-
linux-gnu

*# SOURCE_DATE_EPOCH pinned to the tag commit's author
timestamp,*

per MVP2_SHARED_CORE.md §5.1

export SOURCE_DATE_EPOCH=\$(git log -1 --format=%at "\$
\${github.ref_name}")

- name: Diff artifact hash against primary build

run: |

PRIMARY_HASH=\$(gh release download "\$
\${github.ref_name}" --pattern "*.tar.gz.sha256" -O -)

REBUILD_HASH=\$(sha256sum target/x86_64-unknown-linux-gnu/
release/helix-vpn)

if ["\$PRIMARY_HASH" != "\$REBUILD_HASH"]; then
echo "::error::Reproducibility check FAILED for \$
\${github.ref_name} — artifact hashes differ"
exit 1
fi

Downstream promotion job depends on this one:

promote-to-canary-1:

needs: [verify-reproducibility, sbom-diff-gate]

Honest boundary: reproducibility verification proves the shipped binary was genuinely built from the reviewed, tagged source — it does not by itself prove the binary is defect-free or that the canary rollout will be healthy; those are §10.2's job. The two gates are independent and both required.

11. Appendices

Appendix A: Rust Crate Selection

Crate	Purpose	Version	Recommendation
tokio	Async runtime	^1.35	Required
boringtonun	WireGuard in Rust	^0.6	Recommended
rustls	TLS implementation	^0.23	Recommended
aws-lc-rs	Crypto backend (FIPS, PQ)	^1.6	Recommended
x25519-dalek	X25519 key exchange	^2.0	Required
ed25519-dalek	Ed25519 signatures	^2.0	Required
chacha20poly1305	AEAD cipher	^0.10	Required
aes-gcm	AES-GCM cipher	^0.10	Required
bytes	Zero-copy buffers	^1.5	Required
serde	Serialization	^1.0	Required
zeroize	Secure memory clearing	^1.7	Required
secrecy	Secret types	^0.8	Recommended
hickory-dns	DNS resolver (DoT/DoH/DoQ)	^0.24	Recommended
ipnet	IP network types	^2.9	Recommended
socket2	Advanced socket options	^0.5	Recommended
shadowsocks-rust	Shadowsocks protocol	^1.24	Fallback protocol
quiche	QUIC/HTTP3	^0.20	MASQUE transport

Appendix B: Glossary

Term	Definition
AEAD	

Term	Definition
	Authenticated Encryption with Associated Data - Encryption mode providing both confidentiality and authenticity
DAITA	Defense Against AI-based Traffic Analysis - Traffic shaping to prevent ML-based detection
DPI	Deep Packet Inspection - Advanced network traffic analysis for protocol identification
HNDL	Harvest Now, Decrypt Later - Attack storing encrypted traffic for future quantum decryption
KEM	Key Encapsulation Mechanism - Post-quantum key exchange method
ML-KEM	Module-Lattice-based KEM - NIST post-quantum standard (formerly Kyber)
MASQUE	Multiplexed Application Substrate over QUIC Encryption - IETF proxy standard
NE	Network Extension - Apple's VPN framework (iOS/macOS)
PFS	Perfect Forward Secrecy - Key compromise doesn't expose past sessions
PQ	Post-Quantum - Cryptography resistant to quantum computer attacks
RASP	Runtime Application Self-Protection - In-app tamper detection
SPKI	Subject Public Key Info - Format for public key pinning
STRIDE	Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation - Threat model
WFP	Windows Filtering Platform - Windows firewall API
XTLS	X Transport Layer Security - Optimized TLS proxy protocol

Appendix C: Security Checklist by Platform

Desktop (Windows/macOS/Linux)

#	Feature	Priority	Status
1	Kill switch (system-level)	CRITICAL	[]
2	DNS leak protection	CRITICAL	[]
3	IPv6 leak protection	CRITICAL	[]
4	WebRTC leak protection	HIGH	[]
5	Secure key storage	CRITICAL	[]
6	Certificate pinning	HIGH	[]

#	Feature	Priority	Status
7	Auto-connect on untrusted Wi-Fi	MEDIUM	[]
8	Split tunneling (secure)	MEDIUM	[]
9	Code signing verification	HIGH	[]
10	Crash reporting (no PII)	MEDIUM	[]
11	Update signature verification	CRITICAL	[]
12	Anti-debug/tamper detection	MEDIUM	[]
13	Post-quantum key exchange	LOW	[]
14	Perfect forward secrecy	CRITICAL	[]

Mobile (iOS/Android)

#	Feature	Priority	Status
1	Kill switch (system-level)	CRITICAL	[]
2	DNS leak protection	CRITICAL	[]
3	IPv6 leak protection	CRITICAL	[]
4	Secure key storage	CRITICAL	[]
5	Certificate pinning	HIGH	[]
6	On-demand VPN rules	HIGH	[]
7	Jailbreak/root detection	MEDIUM	[]
8	Biometric auth for VPN keys	MEDIUM	[]
9	Split tunneling (per-app)	MEDIUM	[]
10	Background execution	CRITICAL	[]
11	Update via official stores	HIGH	[]
12	Certificate transparency	MEDIUM	[]
13	Lock screen VPN control	LOW	[]
14	Battery-optimized keepalive	HIGH	[]

Appendix D: Incident Response Plan

Severity	Response Time	Action	Examples
Critical	< 1 hour	Emergency patch, force update	Remote code execution, key extraction
High	< 4 hours	Fast-track patch, staged rollout	Kill switch bypass, DNS leak
Medium	< 24 hours	Normal patch cycle	UI security issue, config leak
Low	< 7 days		

Severity	Response Time	Action	Examples
		Next scheduled release	Documentation issue, non-exploitable bug

Document generated from comprehensive research including 30+ independent web searches across academic papers, vendor documentation, open-source projects, and authoritative technical sources. All citations use [^number^] format referencing original search results.

Last Updated: 2025-07-08 (Revision 1 baseline); Revision 2 hardening pass: 2026-07-04 Next Review: 2025-10-08